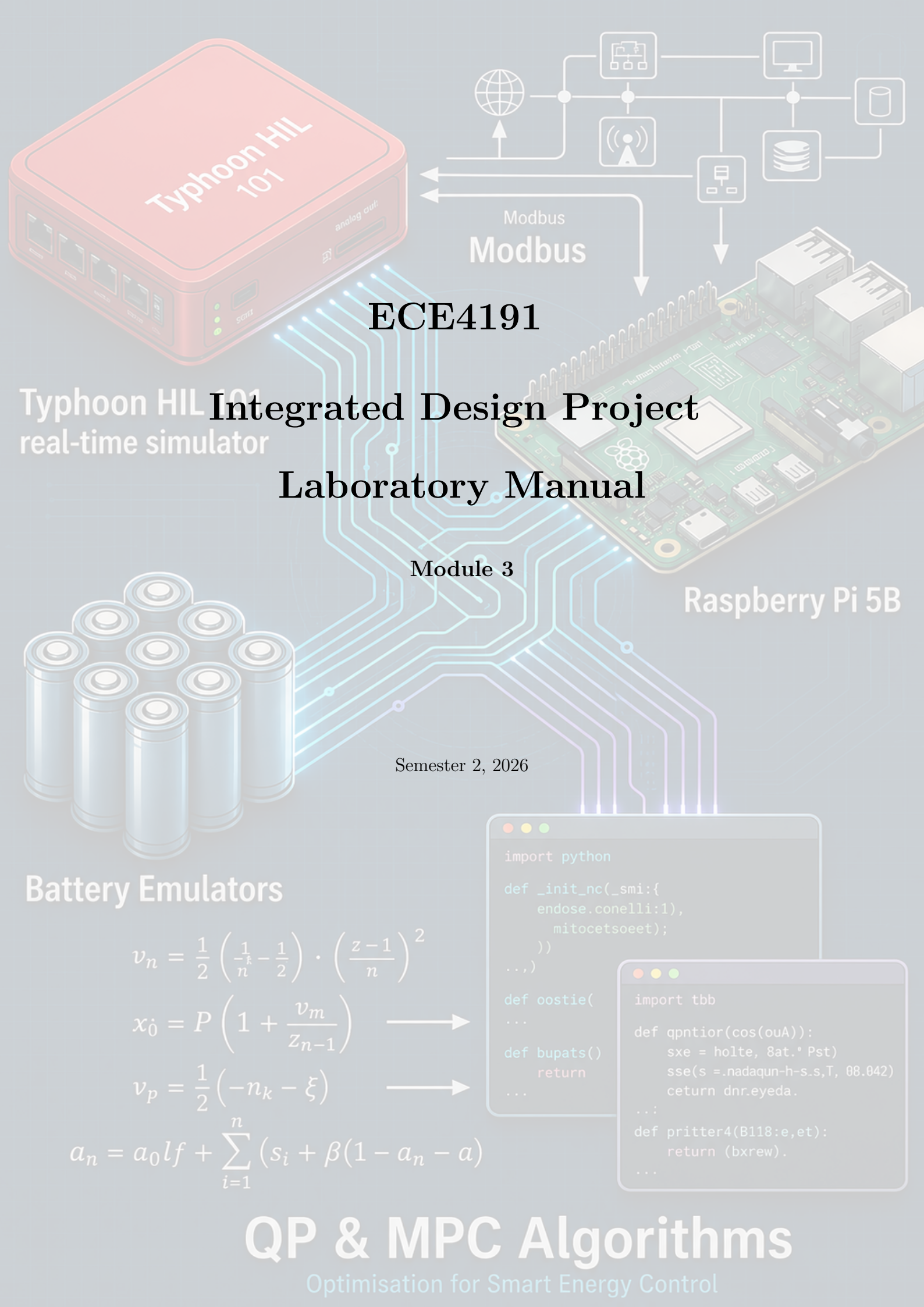


ECE4191

Integrated Design Project

Laboratory Manual

Module 3

Semester 2, 2026

Contents

1 Introduction and Objectives **4**

2 Module 3 System Architecture **7**

3 Battery Disaggregation Across the MIEEE-13NF **9**

4 Raspberry Pi Setup **11**

5 Modbus Connection Setup **14**

6 Modbus Register Mapping **17**

7 Run QP/MPC Algorithms on the Raspberry Pi **20**

8 Assessment (10% of your final grade for ECE4191) **33**

A SSH and File Transfer Workflow **39**

B Measuring Feeder Power at Node 632 using a Power Meter **41**

C Troubleshooting: Raspberry Pi Not Appearing in Raspberry Pi Connect **46**

This document will be revised throughout the semester and the revision history will be summarised below.

Table 1: Revision History

Version	**Date**	**Description of Changes**
v1.0	18/08	Initial release
v1.1	20/08	Raspberry Pi Setup Alternative on P-13 and Appendix. Updated the csv file name in the python code (Section C)
v1.2	25/08	Clarification on playback period on P-34, File names on P-40. Rearranged parts of the assessment.

1 Introduction and Objectives

Welcome to **Module 3** of ECE4191 Integrated Design Project. In Module 1, you set-up the MIEEE-13NF in Typhoon HIL and replayed load and photovoltaic generation profiles. In Module 2, you studied optimisation-based energy shifting using Quadratic Programming (QP) and Model Predictive Control (MPC). In Module 3, we bring together the ideas explored in Modules 1 and 2 by implementing a controller-in-the-loop experiment with a Raspberry Pi that communicates with the Typhoon HIL101.

The aim of this module is to connect the **Typhoon HIL101** to a **Raspberry Pi** controller using the **Modbus TCP/IP** communication protocol. You will implement the QP and MPC algorithms developed in Module 2 on the Raspberry Pi, and investigate the behaviour of the controller on the MIEEE-13NF. For example, does the MPC controller correct the voltage violations observed in Module 1?

Learning Expectations

- Implement **QP-based battery scheduling** and **MPC-based battery control** using the MIEEE-13NF.
- Implement and execute a **Controller-in-the-Loop MPC** experiment at Node 646 incorporating real-time feedback of the measured battery SoC from the Typhoon HIL 101 via **Modbus TCP/IP**.
- Analyse historical measurements from your real-time Typhoon HIL 101 experiments using the MIEEE-13NF, including CIL experiments.
- Compare the **no-control, QP, MPC and MPC-CIL** experiments using the MIEEE-13NF and explain your observations in terms of voltage violations, reverse power flow, and peak demand across the feeder.

Notation Mapping: Module 2 → Module 3

Description	Module 2	Module 3
At time t, average load over a 30 min interval from time step t to $t+1$ in kW. Day-ahead step $k=1$.	$l(t+1\|t)$	$P_{\text{load}}(t)$
Historical/measured (e.g., in Typhoon) load profile over a day	–	P_{load}
Day-ahead load forecast	l	$\hat{P}_{\text{load}}$
At time t, day-ahead load (in kW) across a 30 min interval from day-ahead step $k-1$ to k	$l(t+k\|t)$	$\hat{P}_{\text{load}}(k)$
At time t, average PV generation over a 30 min interval from time step t to $t+1$ in kW. Day-ahead step $k=1$.	$g(t+1\|t)$	$P_{\text{PV}}(t)$
Historical/measured (e.g., in Typhoon) PV generation profile over a day	–	P_{PV}
Day-ahead PV generation forecast	g	$\hat{P}_{\text{PV}}$
At time t, day-ahead PV generation (in kW) across a 30 min interval from day-ahead step $k-1$ to k	$g(t+k\|t)$	$\hat{P}_{\text{PV}}(k)$
At time t, battery charge/discharge rate over 30 mins from time step t to $t+1$ in kW. Day-ahead step $k=1$.	$x_1(t+1\|t)$	$P_{\text{bat}}(t)$
Historical/measured (e.g., in Typhoon) battery (BESS) charge/discharge profile over a day	–	P_{bat}
At time t, day-ahead battery charge/discharge rate (in kW) across a 30 min interval from day-ahead step $k-1$ to k	$x_1(t+k\|t)$	$P_{\text{bat}}(k)$
At time t, average power flowing to and from a grid node over a 30 min interval from time step t to $t+1$ in kW. Day-ahead step $k=1$.	$x_2(t+1\|t)$	$x_2(t)$
Historical/measured (e.g., in Typhoon) power flows to/from a grid node over a day	–	x_2
At time t, forecast of the power flows to/from grid node (in kW) across a 30 min interval from day-ahead step $k-1$ to k	$x_2(t+k\|t)$	$\hat{x}_2(k)$
Day-ahead forecast of the power flows to/from a node	x_2	$\hat{x}_2$
Baseline net load over a 30 min interval from time step t to $t+1$ in kW ($C=0$). Day-ahead step $k=1$.	$\tilde{x}_2(t+1\|t)$	$\tilde{x}_2(t)$
At time t, battery SoC at day-ahead step $k-1$ in kWh	$z(t+k\|t)$	SoC(k)
Battery initial SoC at time step t in kWh	$z(t\|t)$.	SoC(t)
At time t, reactive power to/from the load over a 30 min interval in kVar	–	$Q_{\text{load}}(t)$
Time interval between step $k-1$ and k, and step t and $t+1$	Δ	Δ
Battery capacity in kWh	C	C
Inverter rating (battery charge/discharge limit) in kW	B_{ch}, B_{dis}	B_{ch}, B_{dis}
Day-ahead net metering tariff / time-of-use price	η	η

Module 3 Notation: Real-time Signals, Historical Trajectories, Forecast Signals

Real-time Typhoon HIL Input/Output Signals From Time t to $t+1$	**Historical Trajectory/ Typhoon HIL Measurement Vector**	**Forecast Signals Over a 30 min Interval From Day-ahead Step $k-1$ to k**
$P_{\text{load}}(t)$	P_{load}	$\hat{P}_{\text{load}}(k)$
$P_{\text{PV}}(t)$	P_{PV}	$\hat{P}_{\text{PV}}(k)$
$P_{\text{bat}}(t)$	P_{bat}	$P_{\text{bat}}(k)$
x_2(t)	x_2	$\hat{x}_2(k)$
$\tilde{x}_2(t)$	$\tilde{x}_2$	$\tilde{x}_2(k)$
SoC(t)	SOC	SoC(k)
$Q_{\text{load}}(t)$	Q_{load}	$Q_{\text{load}}(k)$
–	η	$\eta(k)$

Module 3 Notation: Typhoon HIL Inputs/Outputs, Raspberry Pi Inputs/Outputs

Real-time Typhoon HIL input/output signals from time step t to $t+1$	**Real-time Raspberry Pi input/output signals, for all day-ahead steps $k \in \mathcal{S}$, where $\mathcal{S} := \{1, 2, \ldots, s\}$**
Input: $P_{\text{load}}(t)$	QP/MPC Input: $\hat{P}_{\text{load}}(k)$ for all $k \in \mathcal{S}$
Input: $P_{\text{PV}}(t)$	QP/MPC Input: $\hat{P}_{\text{PV}}(k)$ for all $k \in \mathcal{S}$
Input: $Q_{\text{load}}(t)$	QP/MPC Input: $\eta(k)$ for all $k \in \mathcal{S}$
	QP/MPC Input: SoC(k) for $k = 0$
Output: x_2(t) or $\tilde{x}_2(t)$	
	QP Output: $P_{\text{bat}}(k)$ for all $k \in \mathcal{S}$
Control Input: $P_{\text{bat}}(t)$	QP Output: $\hat{x}_2(k)$ for all $k \in \mathcal{S}$
Control Output: SoC(t)	
	MPC Output: $P_{\text{bat}}(k)$ for $k = 1$
	Calculated Output: $\tilde{x}_2(k)$ for all $k \in \mathcal{S}$
	Calculated Output: SoC(k) for all $k \in \mathcal{S}$
	Output: $Q_{\text{load}}(k)$ for $k = 1$

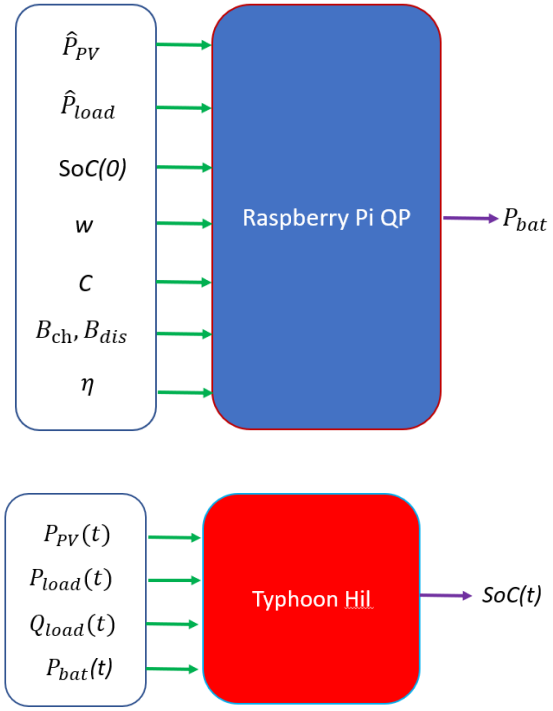


Figure 1: Inputs and outputs for the Raspberry Pi QP algorithm and the Typhoon HIL 101. Inputs to the Raspberry Pi QP algorithm include the day-ahead forecasts of the load $\hat{P}_{load}$ and PV generation $\hat{P}_{PV}$, and the design parameters $SoC(0)$, w, C, B_{ch}, B_{dis}, η. Note that w is a QP-based objective function weight (as defined in Module 2). The output of the Raspberry Pi QP algorithm is the day-ahead battery charge/discharge trajectory P_{bat}. Real-time inputs to the Typhoon HIL 101 are the real-time signals $P_{PV}(t)$, $P_{load}(t)$, $P_{bat}(t)$, $Q_{load}(t)$ and the Typhoon HIL 101 real-time output is the measured $SoC(t)$.

2 Module 3 System Architecture

Figure 2 shows the Module 3 communication architecture, and Figure 1 depicts the inputs and outputs for the Raspberry Pi QP-based algorithm and the Typhoon HIL 101. The Raspberry Pi look-up table reads to the Typhoon HIL 101 the real-time load and PV signals ($P_{\text{load}}(t)$, $P_{\text{PV}}(t)$) —these signals are the same signals you imported in Module 1. The Raspberry Pi also executes the student-developed QP and/or MPC algorithm to compute the battery-based power command, $P_{\text{bat}}(t)$, and writes the battery control signal to the Typhoon HIL Monash IEEE 13 Node Feeder (MIEEE-13NF) using Modbus holding registers. The Typhoon HIL MIEEE-13NF executes the battery-based commands from the Raspberry Pi in real time. See Figure 13 and Figure 14 for more details.

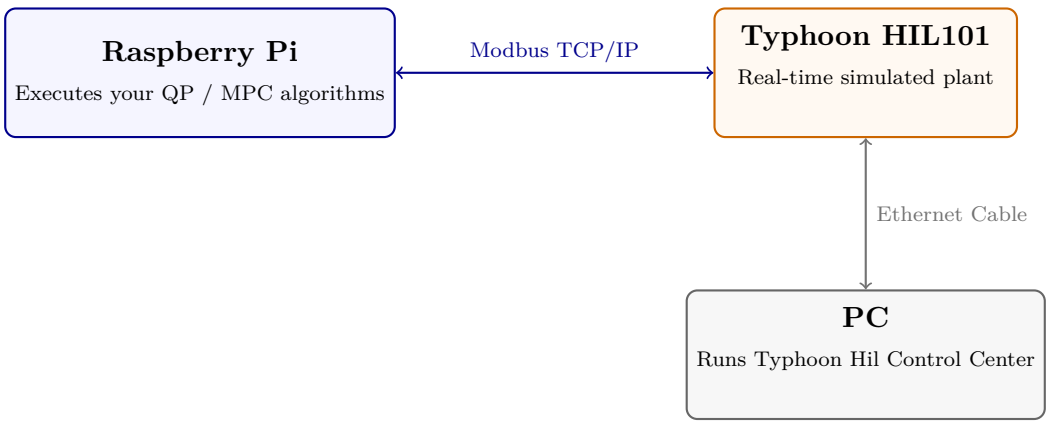


Figure 2: The final interface between all 3 devices.

What is new in Module 3?

The key difference between Module 1 and Module 3 is that in Module 3 we connect an external battery-based controller, the Raspberry Pi, to the Typhoon HIL101.

- In Module 1, the load and PV playback script was launched from the Windows computer using the SCADA Panel Initialization code and the Typhoon HIL Python API. In that workflow, both the script and the HIL simulation were navigated via a Windows computer.
- In Module 3, a Windows computer is used to run the HIL SCADA interface, start and stop the simulation, and export measurements collected during your experiments (e.g., voltage and power measurements from the MIEEE-13NF). That is, the Raspberry Pi will now run your load and PV playback script using a **Modbus TCP/IP** over Ethernet. Specifically, the Raspberry Pi will host a look-up table of the real-time load ($P_{\text{load}}(t)$) and PV generation ($P_{\text{PV}}(t)$) signals to be applied to the MIEEE-13NF during a real-time experiment.
- In Module 3, you will deploy your QP and MPC algorithms (as developed in Module 2) on the Raspberry Pi. Your battery-based power commands ($P_{\text{bat}}(t)$) will be read into the Typhoon HIL101 through **Modbus TCP/IP** over Ethernet.

3 Battery Disaggregation Across the MIEEE-13NF

In Module 2 you implemented a QP-based battery charge/discharge schedule and an MPC-based battery charge/discharge control algorithm. You considered one equivalent aggregated battery representing all the batteries of the 1330 customers connected to the MIEEE-13NF. In Module 3, you will disaggregate the aggregated battery charge/discharge control signal $P_{\text{bat}}(t)$ obtained in Module 2 among the 16 node-phase combinations across the Monash IEEE 13-Node Feeder.

In Module 2, at each half-hour time interval, the QP-based battery schedule, or alternatively, the MPC-based battery controller, returned a battery power command, $P_{\text{bat}}(t)$, to be implemented in real-time. This battery charge/discharge command represented the combined charging or discharging behaviour of all batteries across the MIEEE-13NF. In Module 3, we will disaggregate this battery power command according to the number of customers connected to each node-phase combination, assuming each customer is equipped with a 10 kWh battery.

The Typhoon HIL 101 MIEEE-13NF contains 16 separate batteries located at specific nodes, each with its own $P_{\text{bat}}(t)$ input command. Accordingly, the battery power command, $P_{\text{bat}}(t)$, as developed in Module 2 must be disaggregated amongst the 16 nodes before writing the node-specific $P_{\text{bat}}(t)$ command to the Typhoon HIL MIEEE-13NF via Modbus. We call this process 'Battery Disaggregation'.

Files provided to Students Supporting Battery Disaggregation

Disclaimer

The code provided in this module was developed with the help of GenAI. Although the code has been reviewed and tested, it may still contain errors, limitations, or unexpected behaviour. Students are responsible for understanding, verifying, testing, and debugging the code before using it in their work. Please report any identified issues to the teaching team.

1. `central_agg_forecast_data_students.csv` This file contains the look-up table for the MIEEE-13NF total day-ahead load forecast, $\hat{P}_{\text{load}}$, and total day-ahead PV generation forecast, $\hat{P}_{\text{PV}}$, obtained by aggregating measurements across each node. The day-ahead forecasts are used as inputs to the QP-based battery schedule or the MPC-based battery controller. In your project extension (after completing Module 3), you might consider forecasting algorithms to forecast this data rather than simply aggregating measurement data, as done in Module 3.

2. `central_battery_input_1_day_students.csv` This file contains an example day-ahead aggregated battery schedule ($P_{\text{bat}}(k)$ for all $k = 1, \ldots, 48$), following a QP-based algorithm as per Module 2. That is, each battery power command, $P_{\text{bat}}(k)$, represents the combined behaviour of all 1330 customers connected to the MIEEE-13NF before the process of Battery Disaggregation is performed.

3. `disaggregate_battery_students.py` The file contains the disaggregation logic that splits the aggregated battery commands, $P_{\text{bat}}(k)$, into 16 node-based battery power commands.
To execute the disaggregation code, disaggregate_battery_students.py, place the following files in the same folder:

- `disaggregate_battery_students.py`; and
- `central_battery_input_1_day_students.csv`.

Open a terminal or command prompt in that folder and run:

```
python disaggregate_battery_students.py
```

If the code runs successfully, the following message should appear:

```
Wrote Battery_Power_per_node_1_day.csv: 16 nodes x 48 steps
```

The code creates the output file:

`Battery_Power_per_node_1_day.csv`

The output contains one row for each of the 16 nodes and 48 half-hourly values of $P_{\text{bat}}(k)$. The first column identifies the node-phase combination, while the remaining columns contain the disaggregated battery power commands, $P_{\text{bat}}(k)$, at 30-minute intervals, from `00:30` to `00:00` (that is, day-ahead step $k = 1$ corresponds to `00:30`, step $k = 2$ corresponds to `01:00`, and so on.).

Signal Name Mapping to Typhoon

In Module 3, real-time signals that can be read into the MIEEE-13NF are denoted by $P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$. Battery-based control commands that can be sent to the MIEEE-13NF are denoted by $P_{\text{bat}}(t)$. The relevant mappings for these signals in the `MIEEE-13NF` Typhoon HIL schematic are illustrated in Figure 3. When writing values to the Typhoon HIL, the Raspberry Pi should use the signal names shown in the Typhoon HIL schematic as per Table 3.

Module 3 Variable	Typhoon HIL Signal
$P_{\text{load}}(t)$	`Pref<node>`
$Q_{\text{load}}(t)$	`Qref<node>`
$P_{\text{PV}}(t)$	`PVpanel_power<node>`
$P_{\text{bat}}(t)$	`Battery_Power<node>`

Table 2: Mapping between the Module 3 notation and the corresponding signal names in the Typhoon HIL `MIEEE-13NF` schematic.

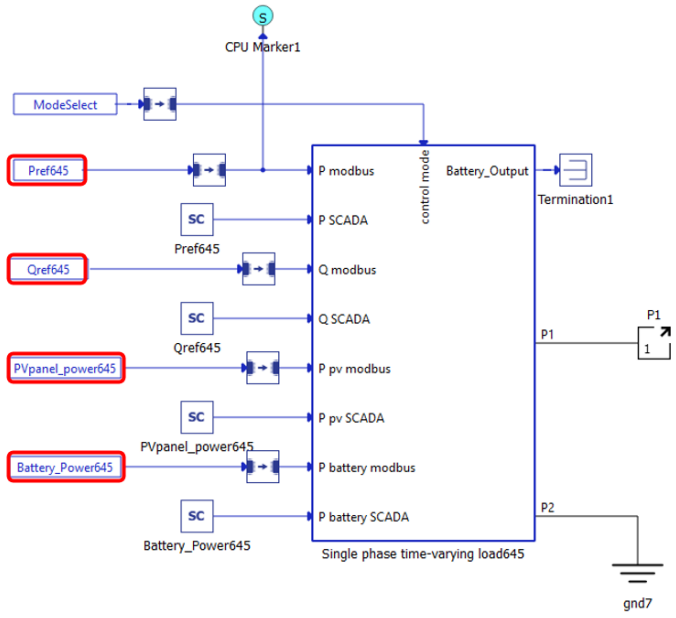


Figure 3: Example signal naming convention used in the Typhoon HIL `MIEEE13NF` schematic.

4 Raspberry Pi Setup

Raspberry Pi Functionalities in Module 3

Note that the real-time load and PV generation data ($P_{\text{load}}(t)$, $P_{\text{PV}}(t)$) to be input into the Typhoon HIL 101 at time t, is included as a look-up table in the CSV file `agg_jan2013_students.csv` that you were given in Module 1, and which you read into the Typhoon HIL through SCADA. In Module 3, you will read this real-time load and PV generation data into the Typhoon HIL 101 through Raspberry Pi. That is, the Raspberry Pi also serves as a database/look-up table by which you can read real-time signals into the Typhoon MIEEE-13NF.

In Module 2, you developed a battery-based controller. Inputs to this controller included day-ahead forecasts of the load ($\hat{P}_{\text{load}}$) and generation ($\hat{P}_{\text{PV}}$). In cases where you assume perfect forecasts (noting the slight abuse of notation), $\hat{P}_{\text{load}} = P_{\text{load}}$ and $\hat{P}_{\text{PV}} = P_{\text{PV}}$. Note that you should not read in forecast values $\hat{P}_{\text{load}}$, $\hat{P}_{\text{PV}}$ to the Typhoon HIL 101 (even for the case of perfect forecast data). Rather, only real-time signals ($P_{\text{load}}(t)$, $P_{\text{PV}}(t)$) and battery commands, $P_{\text{bat}}(t)$, should be read into the Typhoon HIL 101.

In Module 3, the Raspberry Pi serves as a battery-based controller with real-time output signal $P_{\text{bat}}(t)$, as well as a database/look-up table by which we read in to the Typhoon HIL 101 real-time load and PV generation signals ($P_{\text{load}}(t)$, $P_{\text{PV}}(t)$). Importantly, the Raspberry Pi has two functionalities:

1. A database/look-up table of real-time load and PV generation signals ($P_{\text{load}}(t)$, $P_{\text{PV}}(t)$) to be sent to the Typhoon HIL 101 at time t.
2. A battery-based charge/discharge controller that provides commands ($P_{\text{bat}}(t)$) to the Typhoon HIL 101 at time t.

The Typhoon HIL 101 has a battery emulator with a state-of-charge output measurement at Node 646 on the MIEEE-13NF. That is, the other nodes on the MIEEE-13NF that can execute a battery-based power command $P_{\text{bat}}(t)$ do not have a state-of-charge output measurement. Accordingly, we can only perform an MPC-based controller-in-the-loop experiment at Node 646.

For this module, you will perform a controller-in-the-loop (CIL) experiment using the Raspberry Pi. That is, the Raspberry Pi will read in battery SoC measurements from Node 646 in the HIL 101 and will write out battery power commands ($P_{\text{bat}}(t)$) for the battery located in the HIL 101 to follow.

All other nodes on the MIEEE-13NF with time-varying measurements have perfect batteries (i.e., the battery state-of-charge mirrors the estimated value in the MPC or QP-based algorithm). That is, with the exception of Node 646, all other time-varying nodes with a battery power command input $P_{\text{bat}}(t)$ do not have a battery state-of-charge output measurement. As such, in Module 3, you will also perform experiments where the Raspberry Pi writes battery power commands ($P_{\text{bat}}(t)$) to the HIL 101 that are executed without errors in the battery SoC measurement. This functionality does not support a controller-in-the-loop (CIL) experiment, but it is the first step towards such functionality.

Note that it is possible to extend the CIL functionality to all nodes on the MIEEE-13NF with time-varying measurement data as a project extension. For example, such extensions should look at errors in battery SoC measurements vs battery SoC estimates in your QP and MPC algorithm, and the associated grid and customer impacts of such measurement error.

In this section you set up the Raspberry Pi controller to act as the physical controller. Because Eduroam

security protocols block direct peer-to-peer SSH connections, we use a **Personal Hotspot** for the initial configuration.

Step 1: OS Flashing and Headless Configuration

1. Insert the 32 GB Micro SD card into your PC.
2. Open **Raspberry Pi Imager** (v1.9 or later).
3. **Choose Device:** Raspberry Pi 5.
4. **Choose OS:** Raspberry Pi OS (64-bit).
5. **Choose Storage:** Your SD Card. Make sure the **Exclude system drivers** option is checked.
6. **Customization:** In the customization window of **Raspberry Pi Imager**, configure the following settings:
   - **Hostname:** `rpi-team-X` (replace X with your team number).
   - **Localisation:** Navigate to **Localisation** and configure:
     - **Capital city:** `Canberra`
     - **Time zone:** `Australia/Melbourne`
     - **Keyboard layout:** `us`
   - **Username:** `team-X`
   - **Password:** `come up with your team password`
   - **Wireless LAN:** Enter the **SSID** and **Password** of your **Personal Phone Hotspot**.[1]
   - **SSH Authentication:** Navigate to **Remote Access** and enable **SSH**. Select **Use password authentication**, as shown in Figure 4.

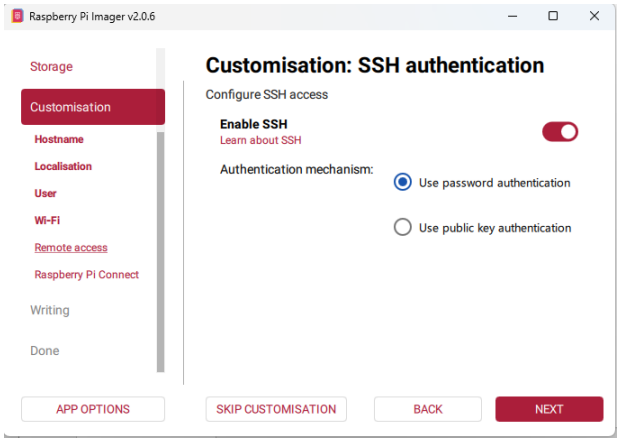
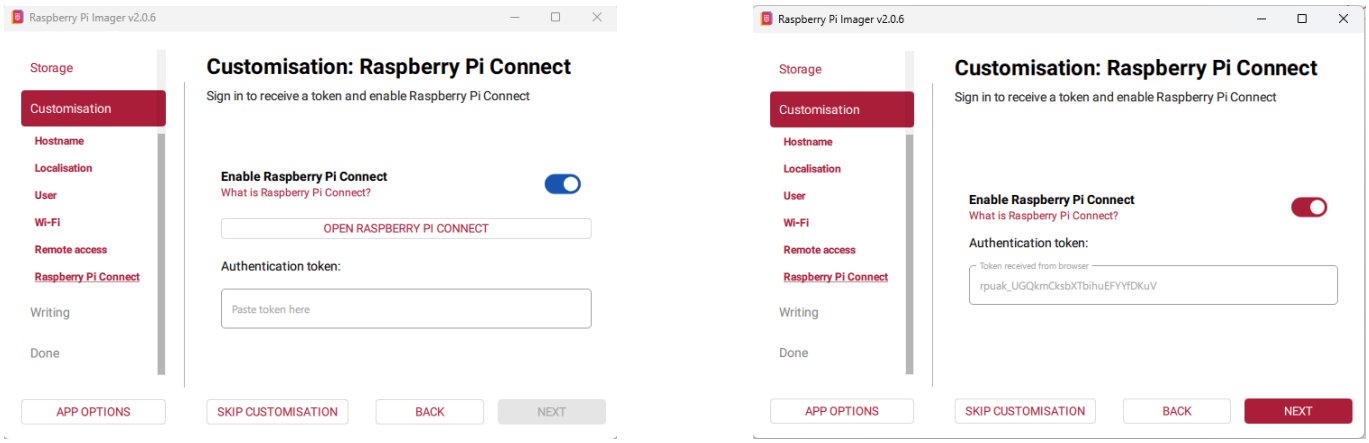


Figure 4: Enabling SSH with password authentication.

   - **Sign in to Raspberry Pi Connect:** Enable **Raspberry Pi Connect** and click **Open Raspberry Pi Connect**, as shown in Figure 5a. This opens a browser window prompting you to sign in to your Raspberry Pi account. After signing in, Raspberry Pi Connect is linked to your device, as shown in Figure 5b.
   - **Create Authentication Key:** Once Raspberry Pi Connect is enabled and the device is linked, click **Create auth key and launch Raspberry Pi Imager**. A new window appears, as shown in Figure 6.

[1] For the SSID and password, use only alphanumeric characters — avoid punctuation, which may be interpreted as a different ASCII value.



(a) Before signing in (b) After signing in

Figure 5: Raspberry Pi Connect interface (a) before and (b) after signing in.

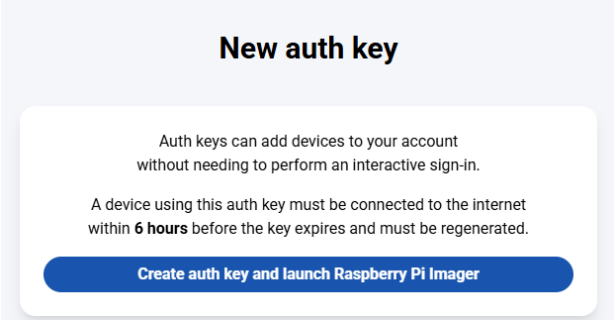


Figure 6: Creating an authentication key after signing in to Raspberry Pi Connect.

- **Write Image:** Click **Next** and then **Write** to flash the OS onto the SD card.

Verify Connection: After inserting the SD card and powering the Raspberry Pi, ensure your hotspot is turned on. Navigate to Raspberry Pi Connect in your browser. The device `rpi-team-X` should appear as **Online**, as shown in Figure 7.

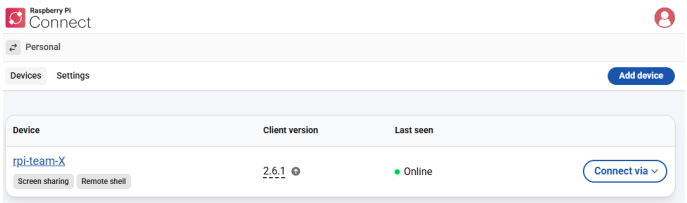
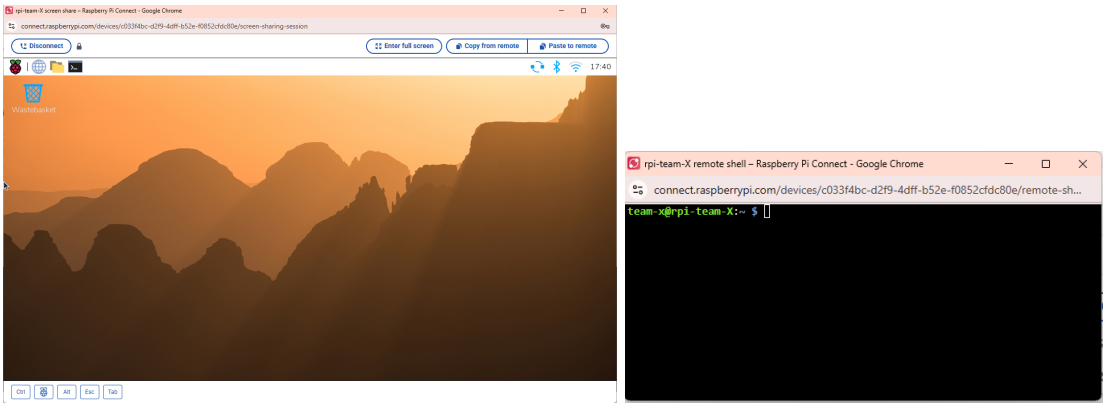


Figure 7: Raspberry Pi device appearing online in Raspberry Pi Connect.

You can connect using either **Screen Sharing** (GUI access) or **Remote Shell** (terminal access), as shown in Figure 8. If the device does not appear in the Raspberry Pi Connect list, or the laptop cannot reach the Pi by hostname, follow the steps in Appendix C.

Note: Two Ways onto the Pi

Raspberry Pi Connect (Screen Sharing / Remote Shell) is used for the initial **configuration** of the Pi over your personal hotspot. For the Module 3 **runs**, you will instead connect the PC and the Pi to the same hotspot and use a direct **SSH** session for command-line control and `scp` for file transfer. The full SSH workflow is given in Appendix A.



(a) Screen sharing interface (b) Remote shell interface

Figure 8: Connecting to the Raspberry Pi via Raspberry Pi Connect: (a) screen sharing and (b) remote shell.

5 Modbus Connection Setup

Modbus is a communication protocol used by industrial devices such as PLCs, sensors, and motor controllers to exchange data. It provides a simple, standardised way to read measurements and send control commands between devices. This section establishes the Ethernet link between the **Typhoon HIL101** and the **Raspberry Pi** via the Modbus protocol. Work through Steps 1–3 to configure both ends of the link, then confirm connectivity with Step 4.

Step 1: Physical Connection

1. Connect an Ethernet cable between **Ethernet Port 2** on the Typhoon HIL101 and the **Ethernet port** on the Raspberry Pi.

Step 2: Configure the Typhoon HIL Network (Ethernet Port 2)

1. Open **Typhoon Hil Control Center** and navigate to **Device Manager**.
2. Right-click the detected HIL device and select **Change ini file**, as shown in Figure 9.

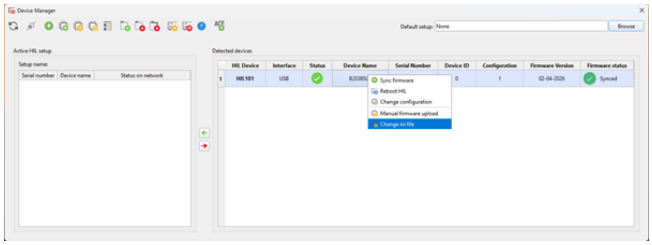


Figure 9: Accessing INI file settings from Device Manager.

3. In the INI configuration window, navigate to **eth_port2** and set:
   - **IP Address:** `192.168.1.40`
   - **Netmask:** `255.255.255.0`
4. Click **Write INI file and reboot HIL** to apply.

Step 3: Configure the Raspberry Pi Network Settings

1. Connect to the Raspberry Pi via **Screen Sharing**.
2. Click the network icon and select `Advanced Options` → `Edit Connections`, as shown in Figure 10.

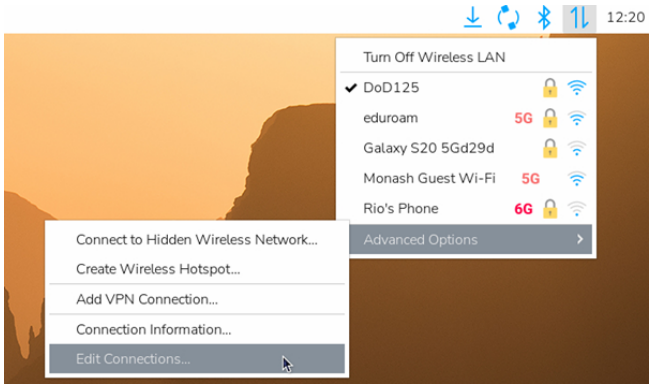
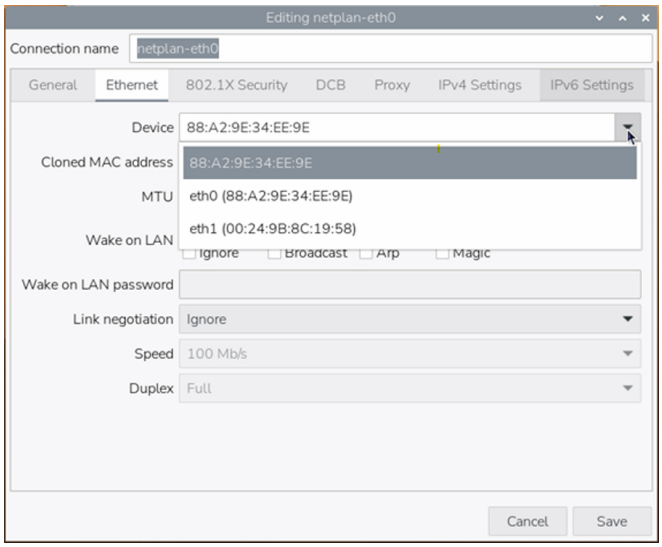
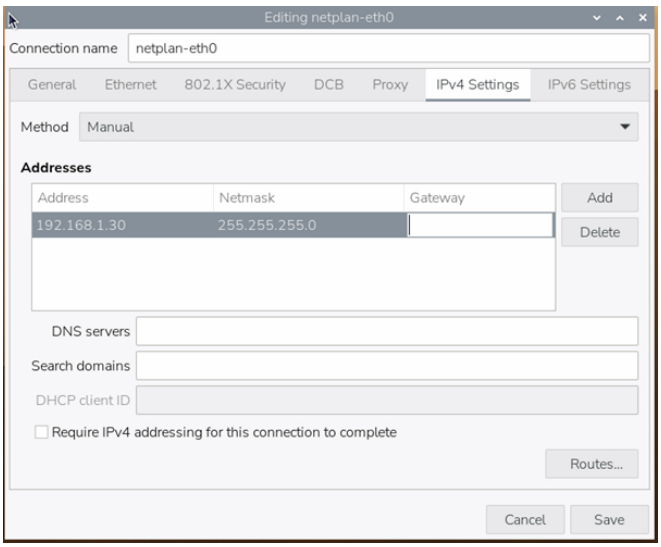


Figure 10: Accessing advanced network settings on the Raspberry Pi.

3. Select the Ethernet interface (e.g. `netplan-eth0`) and click **Edit**, then open the **IPv4 Settings** tab (Figure 11):
   - Set **Method** to **Manual**.
   - **IP Address:** `192.168.1.30`, **Netmask:** `255.255.255.0`, **Gateway:** leave blank.
4. Disable **IPv6 Settings** and click **Save**.



(a) Selecting the Ethernet interface



(b) Manual IPv4 configuration

Figure 11: Raspberry Pi network configuration: (a) selecting the Ethernet interface and (b) configuring IPv4 manually.

Step 4: Verify Connectivity

With the feeder model compiled and running the simulation in HIL SCADA, open a terminal on the Raspberry Pi and ping the HIL Ethernet Port 2:

```
ping 192.168.1.40
```

A response with 0% packet loss confirms the Raspberry Pi can reach the Typhoon HIL over the dedicated Ethernet link. If there is packet loss, please check the raspberry pi setup steps and ensure the ethernet port between the pi and the Typhoon is connected.

Important: Two Separate IP Addresses Are in Use — Do Not Confuse Them

Two distinct IP addresses live on the Typhoon HIL device, and Module 3 depends on telling them apart:

- **192.168.1.40** — the **Network Interface Card (NIC) IP address of Ethernet Port 2**, configured in Step 2 via the Device Manager INI file. It is the hardware-level address that identifies the HIL on the local Ethernet network, and it is the address you `ping` to verify physical connectivity.
- **192.168.1.210** — the **Modbus TCP Server IP address** in the provided `Monash_13NodeFeeder` model. This is the software-level address the Modbus server listens on *while the simulation is running*, and it is the address the QP/MPC controllers on the Raspberry Pi connect to on port `502`.

Both addresses are on the same subnet (`255.255.255.0`), so the Raspberry Pi at `192.168.1.30` can reach both. The feeder model ships pre-configured for `192.168.1.210:502`; you do not need to edit it.

6 Modbus Register Mapping

Why Modbus Registers Are Needed

The Raspberry Pi controller cannot directly modify internal Typhoon HIL schematic variables. Instead, the MIEEE-13NF exposes selected input signals through a **Modbus TCP/IP** server. In Module 3, the Raspberry Pi controller acts as the Modbus client and sends controller outputs to the Typhoon HIL101 by writing to predefined **holding registers**.

The Raspberry Pi can write setpoints for:

- Active power command, $P_{\text{load}}(t)$ (`Pref<node>`);
- Reactive power command, $Q_{\text{load}}(t)$ (`Qref<node>`);
- PV generation command, $P_{\text{PV}}(t)$ (`PVpanel_power<node>`);
- Battery power command, $P_{\text{bat}}(t)$ (`Battery_Power<node>`).

These values are received by the **BusSplitMap** subsystem in the MIEEE-13NF model. BusSplitMap takes the incoming Modbus register values and routes them to the correct feeder setpoint signals.

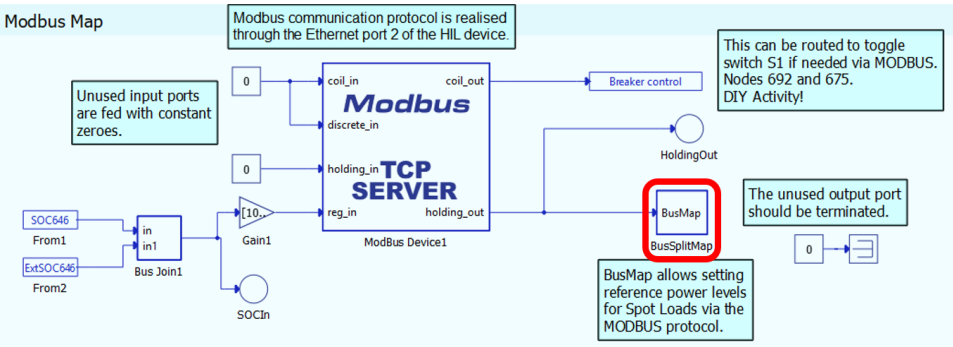


Figure 12: BusSplitMap subsystem used to connect Modbus holding registers to the MIEEE-13NF setpoint signals.

How BusSplitMap Connects Modbus to the MIEEE-13NF Model

The `BusSplitMap` subsystem contains a `Bus Split` block named `ModBusMap`, as shown in Figure 12. This subsystem can be found in the `Monash_13NodeFeeder.tse` schematic. This block has **64 output channels**. Each channel is connected to one feeder setpoint signal.

The 64 holding register addresses are generated in the model initialization script using:

```
gen_holding_addresses = ",".join([f"{i}i" for i in range(2000, 2064)])
```

This creates 64 consecutive integer-type holding registers from **2000 to 2063**. The same script configures the Modbus TCP/IP server used by the Raspberry Pi controller.

Table 3: Modbus TCP/IP configuration for Module 3.

Parameter	Value
Modbus server IP	`192.168.1.210`
Port	`502`
Holding output registers	`2000-2063`
Input registers	`3000, 3001`

Complete BusSplitMap Register Map

Table 4 and 5 show the complete holding register map.

Table 4: Write commands available from the Raspberry Pi controller to the Typhoon HIL101.

Addresses	Node / Phase	Signal order	Direction
2000–2003	Node 646 (Ph B)	$P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$, $P_{\text{bat}}(t)$	Normal
2004–2007	Node 645 (Ph B)	$P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$, $P_{\text{bat}}(t)$	Normal
2008–2011	Node 611 (Ph C)	$P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$, $P_{\text{bat}}(t)$	Normal
2012–2015	Node 652 (Ph A)	$P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$, $P_{\text{bat}}(t)$	Normal
2016–2019	Node 671 (Ph A)	$P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$, $P_{\text{bat}}(t)$	Normal
2020–2023	Node 671 (Ph B)	$P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$, $P_{\text{bat}}(t)$	Normal
2024–2027	Node 671 (Ph C)	$P_{\text{load}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{PV}}(t)$, $P_{\text{bat}}(t)$	Normal
2028–2031	Node 692 (Ph C)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2032–2035	Node 692 (Ph B)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2036–2039	Node 692 (Ph A)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2040–2043	Node 675 (Ph C)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2044–2047	Node 675 (Ph B)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2048–2051	Node 675 (Ph A)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2052–2055	Node 634 (Ph C)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2056–2059	Node 634 (Ph B)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**
2060–2063	Node 634 (Ph A)	$P_{\text{bat}}(t)$, $P_{\text{PV}}(t)$, $Q_{\text{load}}(t)$, $P_{\text{load}}(t)$	**Reversed**

Table 5: Read commands available to Raspberry Pi from the Typhoon HIL101.

Address	Signal	Register name	Direction
3000	Node 646 battery SoC	`SOC646`	HIL101 → Raspberry Pi
3001	Node 646 external SoC	`ExtSOC646`	HIL101 → Raspberry Pi

Why Some Register Orders Are Reversed

The register order is determined by how the BusSplitMap subsystem is wired inside the Typhoon HIL schematic. Some node groups are connected from one side of the Bus Split block, while others are connected from the opposite side. For this reason, the register order for Nodes 692, 675, and 634 are reversed. You are welcome to explore this!

Value Encoding, Scaling, and Sign Conventions

- **Power values are written directly in kW and kVAr.** For example, a load of 230 kW is written as the integer `230`, not `230000`.
- **Registers use signed 16-bit integers.** Each holding register stores a signed integer in the range $[-32768, 32767]$. The provided scripts round the values and clamp them if required before writing to Modbus.
- **Battery sign convention: positive means discharge.** A positive `Battery_Power` value means the battery is discharging and injecting power into the node. A negative value means the battery is charging. The value is written directly to the register without a sign flip.

In the provided Module 3 workflow, only the **Node 646 battery SoC** is read back from the HIL model

through Modbus. Other feeder measurements, such as nodal voltages and measured active powers, are viewed in HIL SCADA and exported manually using the Signal Analyzer.

Table 6: Measurement availability in the Module 3.

Signal or Measurement	**Read through Modbus?**
Node 646 battery State of Charge (SoC)	Yes
Measured node voltages	No
Measured active powers	No

7 Run QP/MPC Algorithms on the Raspberry Pi

Before running a QP-based real-time experiment and an MPC-based CIL experiment, Figures 13 and 14 summarise the corresponding signal flows between the Raspberry Pi controller and the Typhoon HIL 101. A key difference is that the MPC-based CIL implementation uses the measured battery state-of-charge SoC(t) from the HIL as feedback to the Raspberry Pi controller ahead of the next time step.

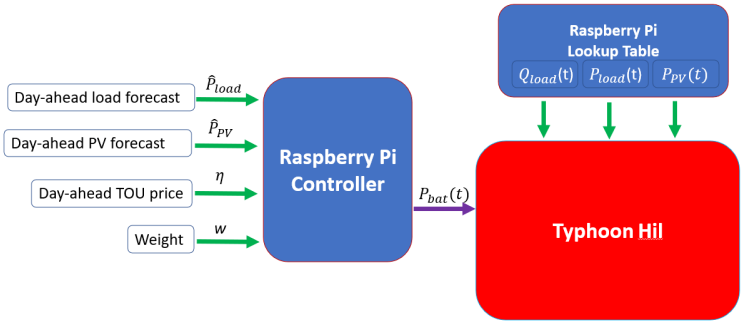


Figure 13: Control block diagram of the Module 3 QP-based implementation.

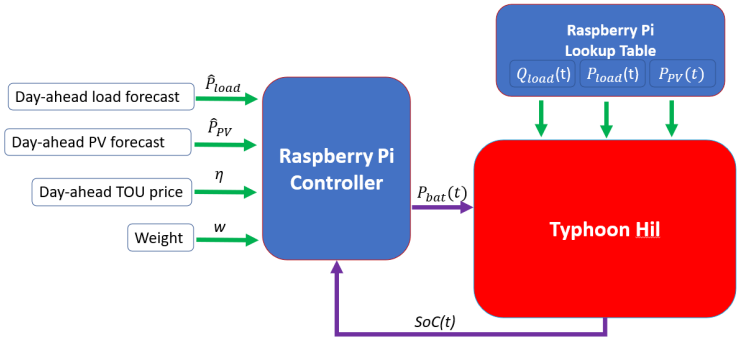


Figure 14: Control block diagram of the Module 3 MPC-based CIL implementation with measured battery state-of-charge feedback.

Disclaimer

The code provided in this module was developed with the help of GenAI. Although the code has been reviewed and tested, it may still contain errors, limitations, or unexpected behaviour. Students are responsible for understanding, verifying, testing, and debugging the code before using it in their work. Please report any identified issues to the teaching team.

In this section, we provide a guided walkthrough of running a toy QP and MPC algorithm for Node 646 on the Raspberry Pi using Modbus TCP/IP. You will:

- At time t, read the Node 646 battery SoC from the Typhoon HIL101 through Modbus.
- Run the QP and MPC algorithms on the Raspberry Pi.
- At time t, send real-time signals $P_{\text{load}}(t)$, $P_{\text{PV}}(t)$, and $P_{\text{bat}}(t)$ to Node 646 on the Typhoon HIL101 through Modbus holding registers.
- After completing the real-time experiment, compare the theoretical battery SoC calculated by the Raspberry Pi controller with the measured SoC from the Typhoon HIL MIEEE-13NF.
- Export measured Node 646 active powers from HIL SCADA using the Typhoon Signal Analyzer.

Files Provided to Students

Transfer the following files to the Raspberry Pi using the procedure provided in Appendix A:

- `toy_qp_modbus_node646_students.py` — an example QP implementation for the Raspberry Pi for Node 646 using Modbus.
- `toy_mpc_modbus_node646_students.py` — an example MPC implementation for Raspberry Pi for Node 646 using Modbus.
- `toy_example_N646_students.csv` — Day-ahead load and PV forecast, across two-days, to be used in this example experiment.
- `actual_N646_students.csv` – Look-up table for real-time load and PV generation signals to be sent to the Typhoon HIL 101 MIEEE-13NF, node 646.

Required Module 3 Change: Panel Initialization

Before running the QP or MPC program, check the HIL SCADA **Panel Initialization** script.
In Module 1, the Panel Initialization code automatically launched a local Python program from the Windows computer when the HIL simulation started. In Module 3, the QP and MPC programs are launched manually from the Raspberry Pi through SSH.
Therefore, the Module 1 Windows auto-launch code must be commented out before starting the QP or MPC run.

```
# import os
# import subprocess

# SETTINGS_DIR = r"C:\path\to\your\Project Directory"
# SCRIPT_PATH = os.path.join(SETTINGS_DIR, "pv_load_windows_typhoon_students.py")
# CSV_PATH = os.path.join(SETTINGS_DIR, "agg_jan2013_students.csv")

# TYPHOON_PYTHON = r"C:\Program Files\Typhoon HIL Control Center 2026.1 sp1\bin\
    typhoon-python.cmd"

# subprocess.Popen(
#     [
#         TYPHOON_PYTHON,
#         SCRIPT_PATH,
#         "--input",
#         CSV_PATH,
#         "--no-prompt"
#     ],
#     cwd=SETTINGS_DIR
# )
```

Listing 1: Panel Initialization code to comment out before Module 3

Prerequisites for the Module 3 Toy Experiment

Before beginning the QP or MPC experiment, confirm that:

1. The Typhoon HIL101 is powered on.
2. The MIEEE-13NF model, `Monash_13NodeFeeder.tse`, has been compiled.
3. The HIL SCADA panel, `Monash_13NodeFeeder.cus`, is open.
4. The Module 1 Panel Initialization auto-launch code has been commented out so that no Windows-based Python program starts automatically.

Step-by-Step Run and Export Procedure for the QP experiment

1. **Confirm that the required files are on the Raspberry Pi.**

   Complete Steps 1–5 in Appendix A to set up the connection and connect to the Raspberry Pi through SSH. Once the SSH connection has been established, navigate to the Module 3 working directory and list its contents:

```
cd ~/toy
ls
```

   Confirm that the following files are present as shown in Figure 15:

   - `toy_qp_modbus_node646_students.py`.
   - `toy_mpc_modbus_node646_students.py`.
   - `toy_example_N646_students.csv`.
   - `actual_N646_students.csv`

   If any of these files are missing, follow Step 7 in Appendix A to transfer them from the Windows computer to the Raspberry Pi. After the transfer is complete, run `ls` again to confirm that all four files are available in the `~/toy` directory.

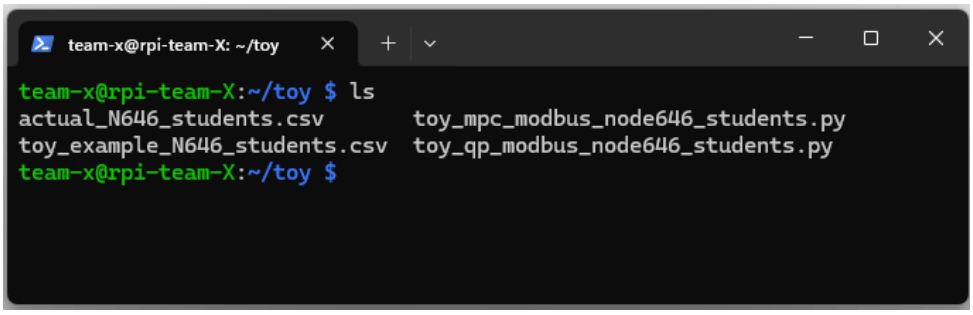


Figure 15: Confirming that the programs and dataset are present on the Raspberry Pi before starting the experiment.

2. **Start the simulation in Remote Control mode.**

   In HIL SCADA, select **Remote Control** and start the simulation. Confirm that the panel is operating in Remote Control mode rather than Local Control mode as shown in Figure 16

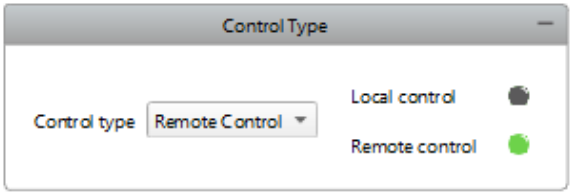


Figure 16: Remote Control mode selected in HIL SCADA before running a program from the Raspberry Pi.

3. **Run the toy QP algorithm from the Raspberry Pi.**

   Prepare the Python Environment on the Raspberry Pi as per Appendix A, Step 8. This step creates the required virtual environment and installs the Python packages used by the QP and MPC algorithms. Once the QP algorithm starts successfully, it displays the experiment settings and a pre-run checklist. The QP algorithm then pauses and prompts you to press **Enter** before beginning the playback, as shown in Figure 17.

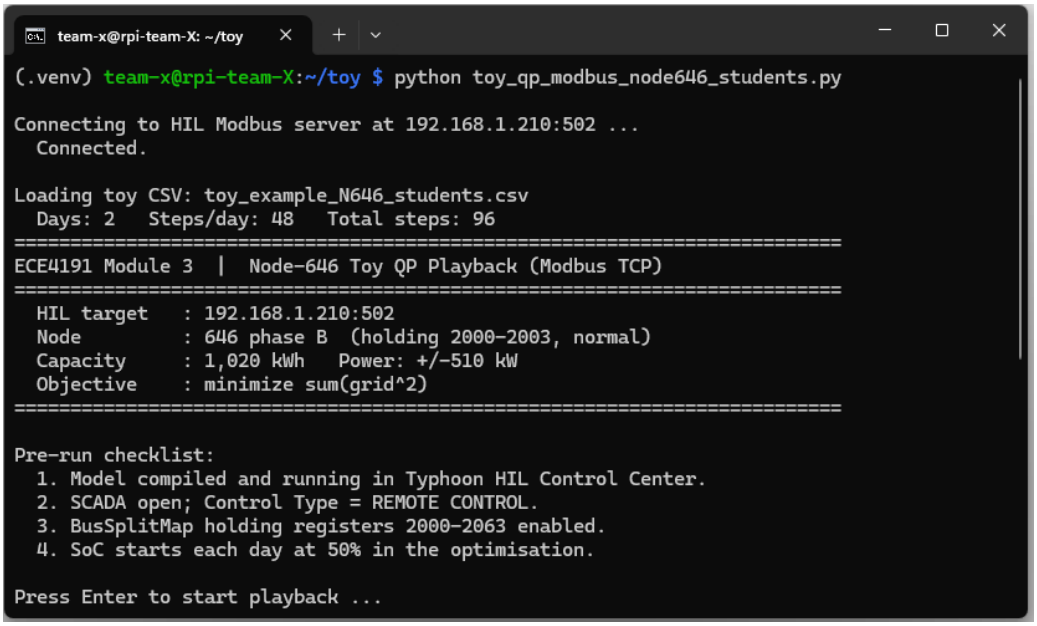


Figure 17: QP algorithm successfully connected to the HIL Modbus server and waiting for the user to press Enter before starting the playback.

Node 646: Toy Forecast vs. Actual Data

Both of the following files contain 96 entries for PV generation and load corresponding to Node 646.

Toy forecast (`toy_example_N646_students.csv`):

- $\hat{\boldsymbol{P}}_{\mathbf{PV}}$ — triangular profile, $0 \rightarrow 600 \rightarrow 0$ kW, repeated over both days.
- $\hat{\boldsymbol{P}}_{\mathbf{load}}$ — held at 300 kW with zero-demand periods and a 700 kW spike to stress-test the controller.

Actual data (`actual_N646_students.csv`):

- $\boldsymbol{P}_{\mathbf{PV}}$ — noisy daytime bell, ≈ 0 kW overnight, peaking near 123 kW.
- $\boldsymbol{P}_{\mathbf{Load}}$ — never zero; overnight baseline $\approx$ 35–50 kW, Day 1 peak $\approx$ 110 kW, Day 2 peak $\approx$ 249 kW.

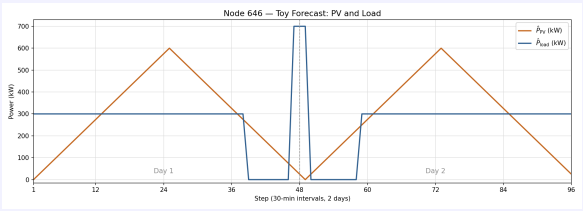


Figure 18: Toy forecast.

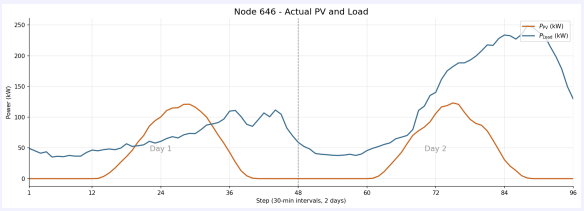


Figure 19: Actual data.

The QP algorithm solves one day-ahead ($s = 48$ steps of duration $\Delta = 30$ min) problem for each of the two days. The QP algorithm includes a daily terminal battery state-of-charge constraint to ensure the battery behaves as an energy shifter and not a net-load or a net-generator.

During the run, the Raspberry Pi executes the following:

(a) Import Node 646 reactive-power real-time signal $Q_{load}(t) = 132$ kVAr, for all t. Set Node 646

battery design parameters $C = 1020$ kWh, $B_{\text{ch}} = 510$ kW, and $B_{\text{dis}} = 510$ kW. Set Node 646 battery initial state-of-charge $SoC(0) = 510$ kWh.

(b) At time t, read in the day-ahead PV generation forecast, $\hat{P}_{\text{PV}}$, and the day-ahead load forecast, $\hat{P}_{\text{load}}$, for Node 646 from `toy_example_N646_students.csv`: an input to the QP algorithm. The file `toy_example_N646_students.csv` contains 96 half-hourly entries, corresponding to two lots of day-ahead forecasts for load and PV generation.

(c) Then solve a day-ahead QP problem for day 1:

$$\min_{P_{bat}(k),\hat{x}_2(k),SoC(k)} \quad \sum_{k=1}^{s} \hat{x}_2^2(k) \tag{1}$$

$$\text{s.t.} \quad \hat{x}_2(k) = \hat{P}_{load}(k) - \hat{P}_{PV}(k) - P_{bat}(k), \quad \forall k \in \mathcal{S}, \tag{2}$$

$$SoC(k) = SoC(k-1) - \Delta\, P_{bat}(k), \quad \forall k \in \mathcal{S}, \tag{3}$$

$$0 \le SoC(k) \le C, \quad \forall k \in \mathcal{S}, \tag{4}$$

$$-B_{\text{ch}} \le P_{bat}(k) \le B_{\text{dis}}, \quad \forall k \in \mathcal{S}, \tag{5}$$

$$SoC(s) = SoC(0). \tag{6}$$

(d) At time t, send to the Typhoon HIL the real-time PV generation, $P_{\text{PV}}(t)$, and the real-time load, $P_{\text{load}}(t)$, from a look-up table in `actual_N646_students.csv`. The look-up table contains 96 half-hourly entries, corresponding to two days of data. Also, send to the Typhoon HIL the real-time battery charge/discharge rate, $P_{\text{bat}}(t)$, obtained from solving the QP problem for day 1, where $P_{\text{bat}}(t) = P_{\text{bat}}(k = 1)$.

(e) More specifically, writes the Node 646 values to holding registers 2000–2003 in the order `[Pref, Qref, PVpanel_power, Battery_Power]`, where `Pref` and `PVpanel_power` are the *actual* real-time load and PV generation entries from `actual_N646_students.csv`, and `Battery_Power` is the optimization-based QP battery command $P_{\text{bat}}(t)$.

(f) Read-in from Typhoon the measured Node 646 battery SoC from Modbus input register 3000 and convert the register value to a percentage by dividing by 100.

(g) At time $t = t+1$, send to the Typhoon HIL the real-time PV generation, $P_{\text{PV}}(t+1)$, and the real-time load, $P_{\text{load}}(t+1)$, from a look-up table in `actual_N646_students.csv`. The look-up table contains 96 half-hourly entries, corresponding to two days of data. Also, send to the Typhoon HIL the real-time battery charge/discharge rate, $P_{\text{bat}}(t+1)$, obtained from solving the QP problem for day 1, where $P_{\text{bat}}(t+1) = P_{\text{bat}}(k = 2)$. Repeat this step until $k = 48$. Then, repeat the above procedure (from step (b)) for day 2.

(h) Clear holding registers 2000–2063 before playback so that only Node 646 is driven by the toy example.

(i) Once the real-time experiment is complete, calculate the power flow to and from Node 646 via the power flow equation below,

$$x_2 = P_{\text{load}} - P_{\text{PV}} - P_{\text{bat}},$$

and compare your calculated x_2 against your recorded real-time measurement data from Node 646 in the Typhoon HIL 101. Also, calculate the baseline power flow to and from Node 646 (when the battery capacity is $C = 0$:

$$\tilde{x} = P_{load} - P_{PV}.$$

(j) Collect real-time experiment data at Node 646 for analysis. Specifically, the day-ahead load ($\hat{P}_{load}$) and PV generation ($\hat{P}_{PV}$) forecasts. The actual/recorded load (P_{load}) and PV generation (P_{PV}) as measured by the Typhoon HIL. The baseline grid power $\tilde{x} = P_{load} - P_{PV}$, the QP-based battery profile (P_{bat}), the optimisation-based grid profile at Node 646 ($\hat{x}_2$), the calculated grid profile at Node 646, $x_2 = P_{\text{load}} - P_{\text{PV}} - P_{\text{bat}}$, and the calculated and measured SoC trajectory. That is, see the timestamped output file `toy_qp_schedule_<timestamp>.csv`.

(k) Then, clear the holding registers after playback.

When the QP run is complete, the program creates one timestamped file:

- `toy_qp_schedule_<timestamp>.csv`.

Use Step 9 in A to download the CSV file from the Raspberry Pi `toy` directory to your Windows machine.

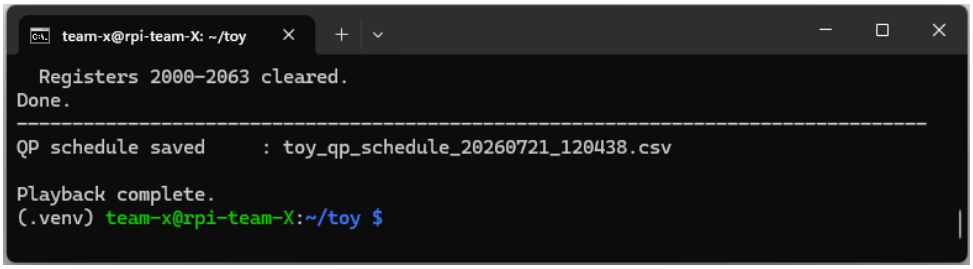


Figure 20: Example of a completed toy QP run on the Raspberry Pi.

Interpreting the Toy QP Output CSV

The file `toy_qp_schedule_<timestamp>.csv` records the QP outputs and the measured battery behaviour (from Typhoon HIL) for every simulation time step t. In this example, the file contains 96 steps, corresponding to two days of 48 half-hour intervals. The battery schedule is computed from the *forecast* data, whereas the grid power is evaluated against the *actual* real-time data imported to Node 646.

- `p_load_fc_kw` and `p_pv_fc_kw` — the day-ahead forecast load, $\hat{P}_{\text{load}}$, and the day-ahead forecast PV generation, $\hat{P}_{\text{PV}}$, for each of the two days, respectively: inputs to the QP algorithm.
- `p_load_kw` and `p_pv_kw` — the measured/recorded load, P_{load}, and PV generation, P_{PV}: Typhoon HIL inputs to Node 646 in the MIEEE-13NF.
- `baseline_grid_kw` — calculated baseline power to and from Node 646 (no control):

$$\tilde{x}_2(t) = P_{\text{load}}(t) - P_{\text{PV}}(t).$$

- `X1_battery_kw` (P_{bat}) — battery charge/discharge trajectory as calculated by the QP. A positive entry means the battery discharged, while a negative entry means the battery charged.
- `battery_action` — description of the battery action: `Charge`, `Discharge`, or `Idle`.
- `X2_grid_kw` — power flow to/from Node 646 as measured in Typhoon, or calculated in Raspberry Pi as follows:

$$x_2(t) = P_{\text{load}}(t) - P_{\text{PV}}(t) - P_{\text{bat}}(t).$$

- `X2_grid_forecast_kw` — QP-based day-ahead forecast of power flowing to and from Node 646. That is, the output of the QP, $\hat{x}_2$, can also be calculated as follows:

$$\hat{x}_2(k) = \hat{P}_{\text{load}}(k) - \hat{P}_{\text{PV}}(t) - P_{\text{bat}}(k), \quad \forall k \in \mathcal{S}$$

- `soc_predicted_pct` — Day-ahead forecast of the SoC, calculated as follows:

$$SoC(k) = SoC(k-1) - \Delta\, P_{bat}(k), \quad \forall k \in \mathcal{S},$$

- `soc_measured_pct` — measured SoC (Typhoon output measurement) from Modbus input register 3000 during the HIL 101 real-time experiment.

The output file `toy_qp_schedule_<timestamp>.csv` can be used to produce plots such as:

- Baseline versus QP-based day-ahead forecast of the power flowing to and from Node 646 — see Figure 22. In Figure 22 we clearly observe the problem of having bad forecast data.

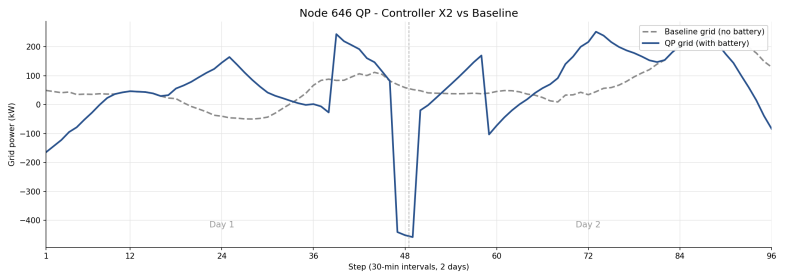


Figure 21: Trajectory of the baseline (no battery) power (in kW) flowing to and from Node 646, ($\tilde{x} = P_{load} - P_{PV}$), versus the measured/reported power flows (with battery) to and from Node 646 (in kW) ($x_2 = P_{\text{load}} - P_{\text{PV}} - P_{\text{bat}}$). Both trajectories cover a two day period as per the real-time simulation experiment.

Same Controller: QP-based Forecast

By way of a comparison to Figure 21, consider Figure 22. In Figure 21 we present the trajectory of the baseline (no battery) power (in kW) flowing to and from Node 646:

$$\tilde{x} = P_{load} - P_{PV}.$$

We also present the trajectory the QP-based day-ahead forecast of the power flows (with battery) to and from Node 646 (in kW), stitched together over the two day period:

$$\hat{x}_2 = \hat{P}_{\text{load}} - \hat{P}_{\text{PV}} - P_{\text{bat}}.$$

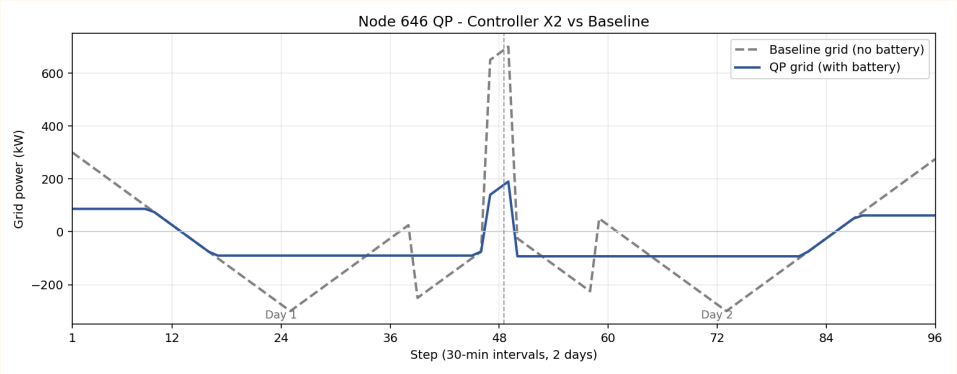


Figure 22: Comparison of the baseline (no battery), $\hat{x}_2$, versus the day-head QP-based power flows, $\hat{x}_2$, to and from Node 646, stitched together over a two day period.

In Figure 22 we observe the day-ahead QP-based power flows flatten the baseline (no battery) trajectory, $\hat{x}_2$, as expected. By contrast, in Figure 21 we observe larger peaks/valleys in power flow $x_2(t)$ as compared to the baseline (no battery) trajectory, a consequence of the large errors in the day-head forecast inputs to the QP-based battery scheduling problem.

4. **Export the Typhoon HIL voltage measurements and active-power measurements using the Signal Analyzer.**

   In the provided Module 3 workflow, the **measured** node voltages and **measured** active powers are not read by the Raspberry Pi through Modbus. These measurements must be exported from the HIL SCADA plots using the Typhoon Signal Analyzer. **Do not click *Stop* in HIL SCADA before exporting the measurements**, as stopping the simulation will clear recorded plot data.

   For both the voltage plots and the active-power plots:

   a. Right-click the plot and select **Export data to Signal Analyzer**, as shown in Figure 23.
   b. Select **Save** in the Signal Analyzer window.
   c. Save the exported measurements in CSV format.

   Use clear filenames, for example:

   - `toy_qp_measured_voltages.csv`;
   - `toy_qp_measured_active_powers.csv`.

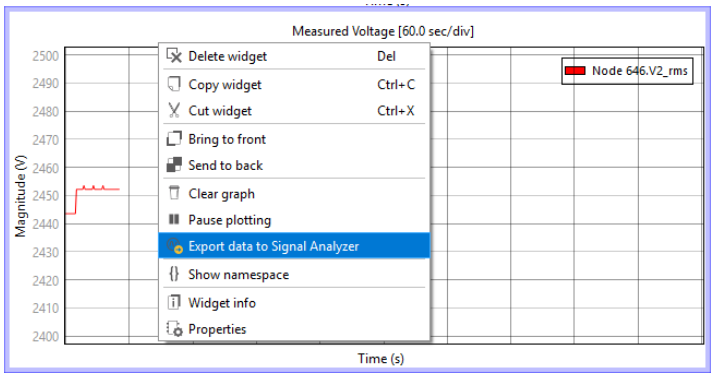
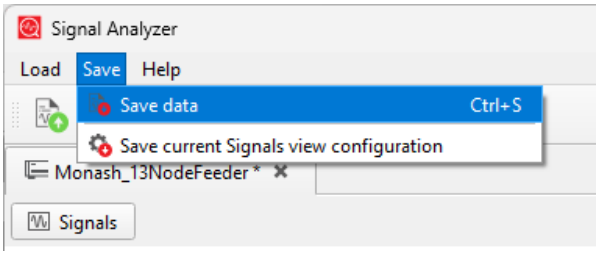
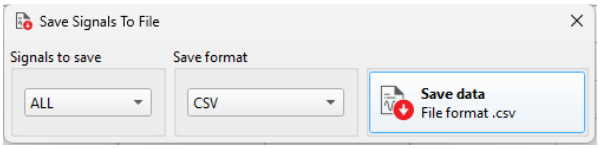


Figure 23: Exporting HIL SCADA plot data to the Typhoon Signal Analyzer.



(a) Saving data from the Signal Analyzer



(b) Selecting the CSV export format

Figure 24: Saving the exported measurements from the Typhoon Signal Analyzer in CSV format.

The `toy_qp_measured_voltages.csv` and `toy_qp_measured_active_powers.csv` files can be used to produce the measured active-power and voltage plots shown in Figures 25 and 26.

Small isolated spikes may appear in the exported signals because of sampling, plot-export timing, or brief transitions between successive setpoints. These spikes may be ignored.

Hint: Mapping Signal Analyzer Samples to Dataset Time Steps

The Signal Analyzer records measurements at approximately **1-second intervals**, while each **30-minute dataset time step** is represented by **2 seconds** of real-time HIL simulation.

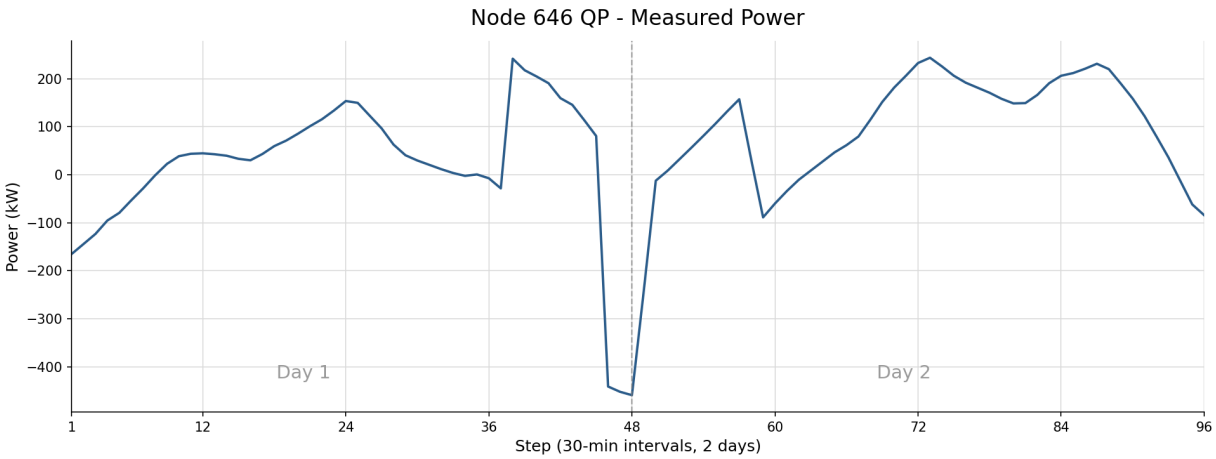


Figure 25: Measured active power at Node 646 during the two-day toy QP experiment.

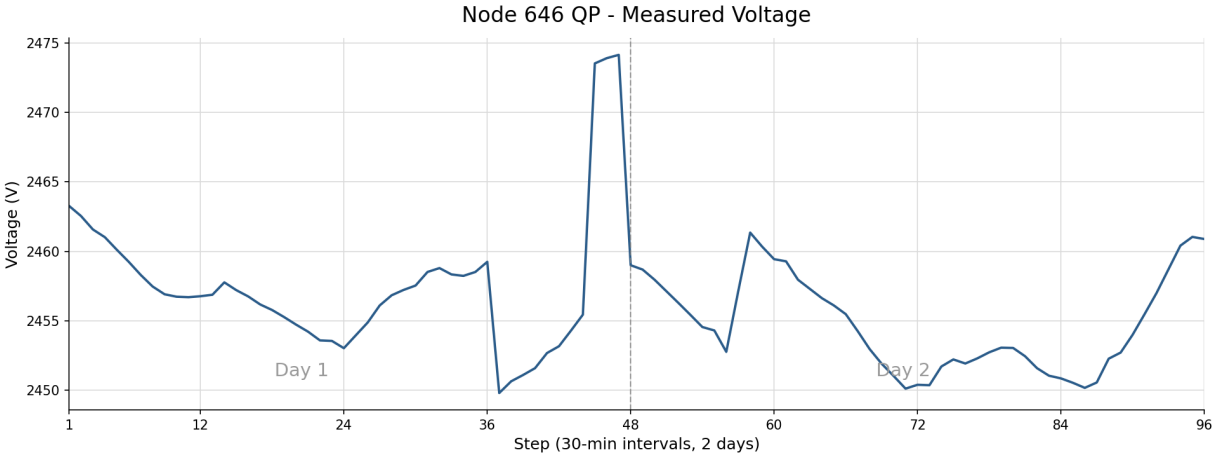


Figure 26: Measured voltage at Node 646 during the two-day toy QP experiment.

Therefore, approximately:

$$\boxed{2 \text{ Signal Analyzer samples} \approx 1 \text{ dataset time step} = 30 \text{ minutes}}$$

When plotting data exported from files such as `toy_qp_measured_voltages.csv` or `toy_qp_measured_active_powers.csv`, take this sampling-rate difference into account.
For example, **100 exported samples correspond to approximately 50 half-hourly dataset steps**.

5. **Stop the QP experiment.**

   After the QP algorithm has finished running and the voltage and active-power measurements have been exported, stop the simulation in HIL SCADA.

   Do not begin the MPC run until the QP simulation has completely stopped.

Step-by-Step Run and Export Procedure for the MPC experiment

1. **Run the toy MPC algorithm from the Raspberry Pi.**

   Restart the HIL simulation in **Remote Control** mode. Then run:

```
cd ~/toy
source .venv/bin/activate
python toy_mpc_modbus_node646_students.py
```

   The MPC algorithm solves a QP-based optimisation problem at every time step. It uses a 48-step look-ahead horizon and applies only the first battery command ($x_1(1)$) before advancing the horizon by one step and solving the optimisation again. Before running the MPC algorithm, we set the design parameters. Specifically, at Node 646 we set the reactive-power to 132 kVAr, the battery capacity to 1020 kWh, the initial SoC of the battery to 510 kWh, and the battery charge/discharge limit to ±510 kW.

   During the MPC run, the Raspberry Pi:

   (a) At time t, read in the day-ahead PV generation forecast, $\hat{P}_{\text{PV}}$, and the day-ahead load forecast, $\hat{P}_{\text{load}}$, for Node 646 from `toy_example_N646_students.csv`: an input to the QP algorithm. The file `toy_example_N646_students.csv` contains 96 half-hourly entries, corresponding to two lots of day-ahead forecasts for load and PV generation.

   (b) At time t, solves a day-ahead QP problem:

$$\min_{P_{bat},\hat{x}_2,SoC} \quad \sum_{k=1}^{s} \hat{x}_2^2(t+k|t) \tag{7}$$

$$\text{s.t.} \quad \hat{x}_2(t+k|t) = \hat{P}_{load}(t+k|t) - \hat{P}_{PV}(t+k|t) - P_{bat}(t+k|t), \quad \forall k \in \mathcal{S}, \tag{8}$$

$$SoC(t+k|t) = SoC(t+k-1|t) - \Delta P_{bat}(t+k|t), \quad \forall k \in \mathcal{S}, \tag{9}$$

$$0 \le SoC(t+k|t) \le C, \quad \forall k \in \mathcal{S}, \tag{10}$$

$$-B_{\text{ch}} \le x_1(t+k|t) \le B_{\text{dis}}, \quad \forall k \in \mathcal{S}, \tag{11}$$

$$SoC(t+s|t) = SoC(t|t) \tag{12}$$

$$SoC(t|t) = \text{SoC}(t). \tag{13}$$

   (c) At time t, send to the Typhoon HIL the real-time PV generation, $P_{\text{PV}}(t)$, and the real-time load, $P_{\text{load}}(t)$, from a look-up table in `actual_N646_students.csv`. The look-up table contains 96 half-hourly entries, corresponding to two days of data. Also, send to the Typhoon HIL the real-time battery charge/discharge rate, $P_{\text{bat}}(t)$, obtained from solving the QP problem (see equations 7-13), where $P_{\text{bat}}(t) = P_{\text{bat}}(t+1|t)$.

   (d) More specifically, writes the Node 646 values to holding registers 2000–2003 in the order `[Pref, Qref, PVpanel_power, Battery_Power]`, where `Pref` and `PVpanel_power` are the *actual* real-time load and PV generation entries from `actual_N646_students.csv`, and `Battery_Power` is the optimization-based QP battery command $P_{\text{bat}}(t)$.

   (e) Advance time step, $t = (t+1)$, read-in from Typhoon the measured Node 646 battery SoC from Modbus input register 3000 and convert the register value to a percentage by dividing by 100. That is, update $SoC(t|t) = SoC(t)$. Specifically, the battery state of charge is measured at the beginning of the next time step (or otherwise, the end of the previous time-step).

   (f) At time step $t = (t+1)$, send to the Typhoon HIL the real-time PV generation, $P_{\text{PV}}(t+1)$, and the real-time load, $P_{\text{load}}(t+1)$, from a look-up table in `actual_N646_students.csv`. The look-up table contains 96 half-hourly entries, corresponding to two days of data.

(g) At time step $t = (t+1)$, read in the updated day-ahead PV generation forecast, $\hat{P}_{\text{PV}}$, and the updated day-ahead load forecast, $\hat{P}_{\text{load}}$, for Node 646 from `toy_example_N646_students.csv`: an input to the QP algorithm. The file `toy_example_N646_students.csv` contains 96 half-hourly entries, corresponding to two lots of day-ahead forecasts for load and PV generation.

(h) At time step $t = (t+1)$, solve a day-ahead QP problem equations 7-13).

(i) At time $t = (t+1)$, send to the Typhoon HIL the real-time PV generation, $P_{\text{PV}}(t)$, and the real-time load, $P_{\text{load}}(t)$, from a look-up table in `actual_N646_students.csv`. The look-up table contains 96 half-hourly entries, corresponding to two days of data. Also, send to the Typhoon HIL the real-time battery charge/discharge rate, $P_{\text{bat}}(t)$, obtained from solving the QP problem (see equations 7-13), where $P_{\text{bat}}(t) = P_{\text{bat}}(t+1|t)$.

(j) Repeat the above steps (e)-(i),

(k) Stop MPC run when $t = t + 48\Delta$

While the MPC algorithm is running, note that Raspberry Pi does the following.

- Check for updates in the day-ahead forecasts $\hat{P}_{load}(t+k|t)$ and $\hat{P}_{PV}(t+k|t)$, for all $k \in \mathcal{S}$, before running the next time-step t.
- Sends to Typhoon Node 646 reactive-power real-time signal $Q_{load}(t) = 132$ kVAr, for all t.
- Updates the initial battery SoC at each time step, for Node 646. The measured initial SoC is read from Modbus input register 3000 before the new battery charge/discharge signal is executed.
- Writes the Node 646 inputs to the Typhoon to holding registers 2000–2003 in the order `[Pref, Qref, PVpanel_power, Battery_Power]`, where `Pref` and `PVpanel_power` are the *real-time* load and PV generation inputs to Typhoon HIL 101 taken from `actual_N646_students.csv`, and `Battery_Power` is the optimisation-based battery commands $P_{\text{bat}}(t)$ as applied at each time-step.
- Converts the measured SoC register value to a percentage by dividing it by 100. If the Modbus SoC reading fails, the controller falls back to a calculated SoC estimate.
- Collect real-time experiment data at Node 646 for analysis. Specifically, the day-ahead load ($\hat{P}_{load}$) and PV generation ($\hat{P}_{PV}$) forecasts. The actual/recorded load (P_{load}) and PV generation (P_{PV}) as measured by the Typhoon HIL. The baseline grid power $\tilde{x} = P_{load} - P_{PV}$, the MPC-based battery profile ($P_{bat}(t)$), the optimisation-based grid profile at Node 646 ($\hat{x}_2$), the calculated grid profile at Node 646, $x_2 = P_{\text{load}} - P_{\text{PV}} - P_{\text{bat}}$, and the predicted SoC, and the measured SoC trajectories. The real-time experimental data is recorded in the timestamped output file `toy_mpc_schedule_<timestamp>.csv`.
- Clears the holding register after the MPC algorithm stops.

When the MPC run is complete, the program creates one timestamped file:

- `toy_mpc_schedule_<timestamp>.csv`.

Use Step 9 in A to download the CSV file from the Raspberry Pi `toy` directory to your Windows machine.

Interpreting the Toy MPC Output CSV

Refer to the **Interpreting the Toy QP Output CSV** colorbox for an explanation of the output-file structure, column names, battery sign convention, and grid-power calculations. The MPC output uses the same file structure and naming conventions.
The output file `toy_mpc_schedule_<timestamp>.csv` can be used to produce plots such as:

- Baseline (no battery) versus MPC-based power flow to and from Node 646 — see Figure 21

for a comparison. That is, we use `baseline_grid_kw` and `X2_grid_kw`, as shown in Figure 27. Specifically, `X2_grid_kw` — power flow to/from Node 646 as measured in Typhoon, or calculated in Raspberry Pi as follows:

$$x_2(t) = P_{\text{load}}(t) - P_{\text{PV}} - P_{\text{bat}}(t).$$

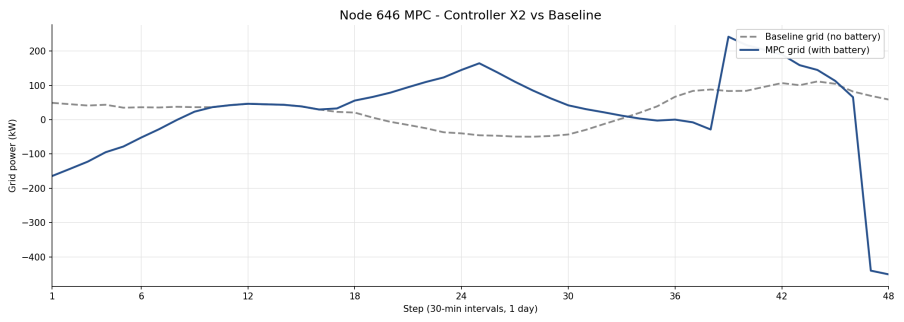


Figure 27: Trajectory of the baseline (no battery) power (in kW) flowing to and from Node 646, ($\tilde{x} = P_{load} - P_{PV}$), versus the measured/reported power flows (with battery) to and from Node 646 (in kW) ($x_2 = P_{\text{load}} - P_{\text{PV}} - P_{\text{bat}}$). Both trajectories cover a one-day period as per the real-time MPC experiment.

Note that our MPC algorithm requires a day-ahead forecast to run. Therefore, we need 2 days of forecast data to run a single day MPC experiment.

Same Controller: MPC-based Forecast

By way of a comparison to Figure 27, consider Figure 28. In Figure 27 we present the trajectory of the baseline (no battery) power (in kW) flowing to and from Node 646:

$$\tilde{x} = P_{load} - P_{PV}.$$

We also present the trajectory the MPC forecast of the power flows (with battery) to and from Node 646 (in kW), stitched together over the two day period:

$$\hat{x}_2 = \hat{P}_{\text{load}} - \hat{P}_{\text{PV}} - P_{\text{bat}}.$$

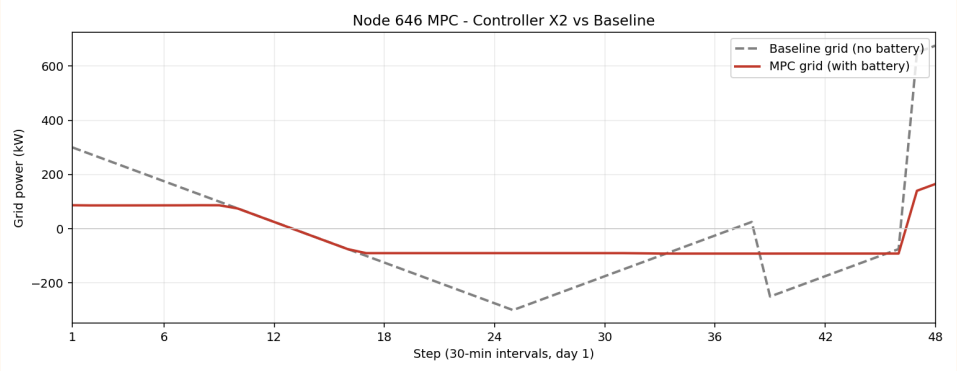


Figure 28: Comparison of the baseline (no battery), $\hat{x}_2$, versus the MPC-based power flows, $\hat{x}_2$, to and from Node 646.

In Figure 28 we observe the MPC-based power flows flatten the baseline (no battery) trajectory, $\hat{x}_2$, as expected. By contrast, in Figure 27 we observe larger peaks/valleys in power flow $x_2(t)$ as compared to the baseline (no battery) trajectory, a consequence of the large errors in the day-head forecast inputs to the MPC-based battery scheduling problem.

2. **Export the MPC voltage and active-power measurements using the Signal Analyzer.**

   Repeat the Signal Analyzer export procedure used for the QP run.

   Use clear filenames, for example:

   - `toy_mpc_measured_voltages.csv`; and
   - `toy_mpc_measured_active_powers.csv`.

   The `toy_mpc_measured_voltages.csv` and `toy_mpc_measured_active_powers.csv`files can be used to produce the measured active-power and voltage plots shown in Figures 29 and 30.

   After exporting the MPC measurements, stop the simulation in HIL SCADA.

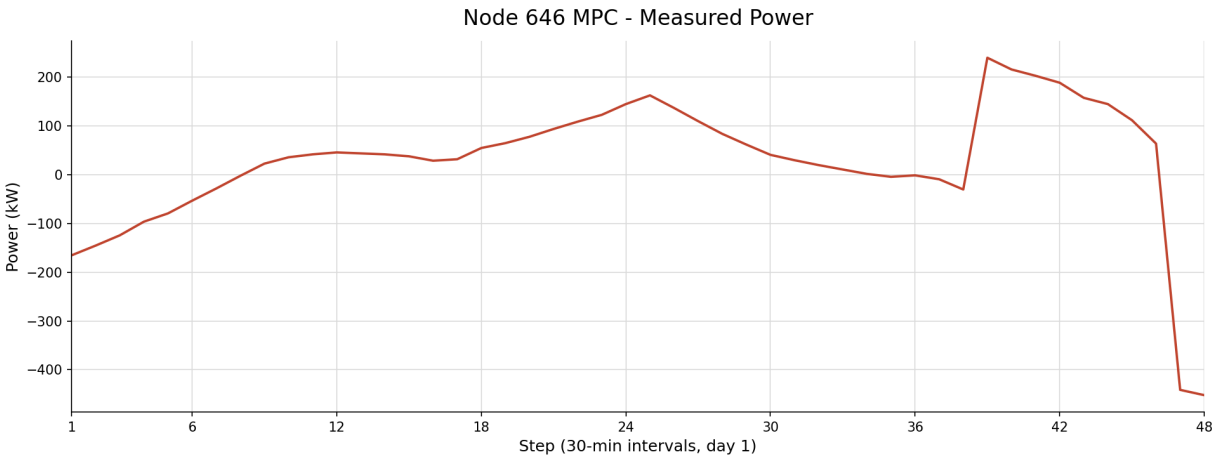


Figure 29: Measured active power at Node 646 during the one-day toy MPC experiment.

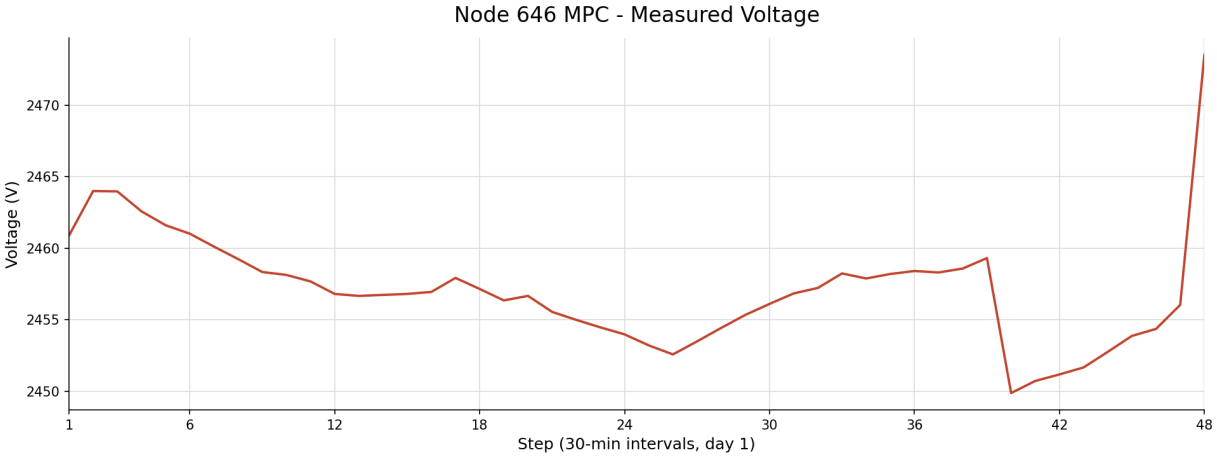


Figure 30: Measured voltage at Node 646 during the one-day toy MPC experiment.

8 Assessment (10% of your final grade for ECE4191)

The Module 3 assessment evaluates your implementation and analysis of three battery-based control experiments using the MIEEE-13NF and your **Raspberry Pi controller** that will communicate via Modbus TCP/IP:

- **Experiment 1 — QP:** Implement the feeder-wide QP-based day-ahead battery scheduling algorithm as developed in Module 2 on the MIEEE-13NF. The QP algorithm will determine the aggregated battery charge/discharge schedule for the entire feeder, which is to be disaggregated across the node-phase battery groups before being applied to the Typhoon HIL 101. The resulting battery charge/discharge commands are then played back over the **5-day playback period**. Prepare the required feeder-wide measurement plots as specified in the following subsection.
- **Experiment 2 — MPC:** Implement the feeder-wide MPC algorithm as developed from Module 2 on the MIEEE-13NF. At each time step t, the MPC algorithm solves a QP-based optimization problem for the day-ahead and applies only the first battery charge/discharge control action to the respective node on the feeder. The aggregated battery command is then disaggregated across the node-phase battery groups before being written to the Typhoon HIL 101. Advance the time step $t = t+1$, repeat, and obtain the battery charge/discharge control signals over the **4-day playback period**. Prepare the required feeder-wide measurement plots as specified in the following subsection.

- **Experiment 3 — MPC-CIL:** Implement a **Controller-in-the-Loop (CIL)** MPC experiment for the battery at Node 646 (Phase B), using the objective function developed in Module 2. The entire MIEEE-13NF remains in operation, with load, PV generation and reactive-power inputs applied across all node-phase combinations. However, only the battery at Node 646 (Phase B) is controlled via the CIL MPC algorithm; the battery commands at all other node-phase combinations are set to zero. At each time step t, the measured Node 646 battery SoC is read from the Typhoon HIL 101 to update the initial state-of-charge input to the MPC algorithm. Note the battery capacity and limits for the battery at Node 646 (Phase B) are provided earlier in Module 3. Prepare the required feeder-wide measurement plots as specified in the following subsection.

Required Plots for the Three Experiments

Before beginning your group-based assessment, students must complete all three experiments and prepare the required plots (pre-work). The required plotting periods differ between experiments: **Experiment 1** (QP) produces results over a 5-day playback period, while, **Experiment 2** (MPC) and **Experiment 3** (MPC-CIL) each produce results over a 4-day playback period. For **each experiment**, export the measured active-power and RMS-voltage data from HIL SCADA using the Typhoon Signal Analyzer and prepare the following plots:

1. **Feeder active power at Node 632** — measured feeder active power over the playback period. Refer to Appendix B.
2. **Active power at all single-phase nodes with time-varying loads** — measured active power at the monitored single-phase node-phase combinations over the playback period (as you did previously in Module 1.) That is, you should have three plots for a three-phase node, one plot representing each respective phase. Likewise, you should have two plots for a two-phase node, one plot representing each respective phase.
3. **Node 634 RMS voltages** — phases A, B and C. Also show the upper and lower voltage limits at ±5% of the 277 V nominal line-to-ground voltage using dashed lines (as you did previously in Module 1.)
4. **RMS voltages at all other monitored feeder nodes** — excluding Node 634. Also show the upper and lower voltage limits at ±5% of the 2401 V nominal line-to-ground voltage using dashed lines (as you did previously in Module 1.)
5. **Measured battery State of Charge at Node 646** — show the measured battery state-of-charge behaviour over the playback period.

These five plots must be prepared separately for: Experiment 1 — QP, Experiment 2 — MPC and Experiment 3 — MPC-CIL.

In addition, prepare the following reference and comparison plots:

- **Module 1 no-control baseline:** measured feeder active power at Node 632. Refer to Appendix B.
- For the following comparisons, use **4-day playback period**:
  - **Node 646 Phase B active-power comparison:** overlay the results from Module 1 no control, Experiment 1 QP, Experiment 2 MPC, and Experiment 3 MPC-CIL on the same axes.
  - **Node 646 Phase B RMS-voltage comparison:** overlay the results from Module 1 no control, Experiment 1 QP, Experiment 2 MPC, and Experiment 3 MPC-CIL on the same axes.

Assessment Workflow

The assessment runs for **20 minutes** and consists of two components:

1. **Pre-work results and oral questions**

   Before attending the assessment, complete **Experiment 1 (QP)**, **Experiment 2 (MPC)** and **Experiment 3 (MPC-CIL)**, and prepare all required plots as specified earlier.

2. **Live demonstration — Experiment 3 (MPC-CIL)**

   During the assessment, run the **4-day Experiment 3 MPC-CIL** implementation at Node 646 (Phase B) using the Raspberry Pi and Modbus TCP/IP. While Experiment 3 is running, you will answer the oral questions relating to the pre-work for Experiment 1, and Experiment 2 using your prepared plots and your pre-prepared observations. After Experiment 3 is finished running, you will answer the oral questions related to Experiment 3.

Grade Sheet

Student Names:
Student IDs:
Final Grade: /100

1. Pre-work preparation: Experiment 1 and Experiment 2 (Grade: /50)
You will provide oral-based answers to your pre-work questions while Experiment 3 is running. No reading from pre-work material allowed. Full marks demonstrate sufficient understanding of the observed feeder behaviour. Each student in the group needs to provide an oral-based answer to be awarded the group mark.

Experiment 1 (Pre-work /24)
QP-based battery schedules read-in to the MIEEE-13NF to all nodes with time-varying loads. This provides a benchmark of centralized battery scheduling.

1. Which nodes violate the upper or lower voltage bounds after QP-based battery scheduling? How do your results compare to your observations in Module 1?
   Grade: /4
2. When do the voltage violations occur after QP-based battery scheduling?
   Grade: /2
3. What is the worst-case voltage violation for each node where a voltage violation occurs after QP-based battery scheduling?
   Grade: /2
4. Which nodes exhibit reverse active power flow after QP-based battery scheduling?
   Grade: /2
5. When does the reverse power flow occur after QP-based battery scheduling?
   Grade: /2
6. What is the largest reverse power flow for each node where reverse power flow occurs after QP-based battery scheduling?
   Grade: /2
7. When is the total feeder net load highest after QP-based battery scheduling?
   Grade: /2
8. What is the peak feeder net load after QP-based battery scheduling?
   Grade: /2
9. How do you measure the peak feeder net load after QP-based battery scheduling?
   Grade: /2
10. Which nodes contribute most to the peak load for the feeder after QP-based battery scheduling? How do your results compare to your observations in Module 1?
    Grade: /4

Experiment 2 (pre-work /26)
MPC-based battery commands read-in to the MIEEE-13NF to all nodes with time-varying loads. This provides a benchmark of centralized control.

1. Which nodes violate the upper or lower voltage bounds after MPC-based battery control? How do your results compare to your observations in Module 1?
   Grade: /2

2. When do the voltage violations occur after MPC-based battery control?
   Grade: /2
3. What is the worst-case voltage violation for each node where a voltage violation occurs after MPC-based battery control?
   Grade: /2
4. Which nodes exhibit reverse active power flow after MPC-based battery control?
   Grade: /2
5. When does the reverse power flow occur after MPC-based battery control?
   Grade: /2
6. What is the largest reverse power flow for each node where reverse power flow occurs after MPC-based battery control?
   Grade: /2
7. When is the total feeder net load highest after MPC-based battery control?
   Grade: /2
8. What is the peak feeder net load after MPC-based battery control?
   Grade: /2
9. How do you measure the peak feeder net load after MPC-based battery control?
   Grade: /2
10. Which nodes contribute most to the peak load for the feeder after MPC-based battery control? How do your results compare to your observations in Module 1?
    Grade: /2
11. How does the peak-load period influence voltages across the feeder after QP-based battery scheduling and MPC-based battery control? How do your results compare to your observations in Module 1?
    Grade: /2
12. Which approach improves voltages across the feeder the most? QP-based battery scheduling or MPC-based battery control? Explain your answer.
    Grade: /4

2. Successful Real-Time CIL Experiment 3 and Pre-Work Plots for Each of the Three Experiments (Grade: /25)

In Experiment 3, the MIEEE-13NF reads in load and PV generation values for all nodes. In Experiment 3, only Node 646 has an operational battery. That is, no battery charge/discharge commands should be read in from the Raspberry Pi, with the exception of battery commands for Node 646. Note that the battery-state-of-charge corresponding to Node 646 should be read into the Raspberry Pi ahead of new battery charge/discharge commands being issued. This provides a benchmark for implementing CIL.

- Demonstrate your real-time Experiment 3, including presentation of the required plots. Grade: /15
- Present the required plots for all three experiments as done in your pre-work. Grade: /10

3. Experiment 3: Oral-based Questions (Real-time CIL Demonstration) (Grade: /25)

Provide oral-based answers to Experiment 3 once your real-time simulation is complete (you cannot read off a script and you must use your real-time simulation results to elaborate on your answers). You can refer to plots from your pre-work for Experiments 1 and 2 and Module 1. Approximate values obtained by visually

inspecting the plots are acceptable. Full marks demonstrate sufficient understanding of the observed feeder behaviour. Each student in the group needs to provide an oral-based answer to be awarded the group mark. Note each question below has a follow-up (secondary) question to support the participation of each student in the group.

1. Consider the MPC-based battery control algorithm implemented in Typhoon MIEEE-13NF at Node 646 over the 4-day period. How is the battery charge/discharge and state-of-charge behaviour different to experiment 2? Explain your answer.
   Grade: /5

2. Consider the MPC-based battery control algorithm implemented in Typhoon MIEEE-13NF at Node 646 over the 4-day period. Is there any battery state-of-charge error introduced by the battery model in the Typhoon MIEEE-13NF at Node 646? Explain your answer.
   Grade: /5

3. Consider the MPC-based battery control algorithm implemented in Typhoon MIEEE-13NF at Node 646 over the 4-day period. What change do you observe in voltages across the feeder with and without the MPC-based battery control algorithm? That is, compare your results to those in Module 1. Explain your answer.
   Grade: /10

4. Consider the MPC-based battery control algorithm implemented in Typhoon MIEEE-13NF at Node 646 over the 4-day period. What change do you observe in voltages across the feeder with the MPC-based battery control algorithm compared to experiment 2? That is, compare your results to those in Experiment 2. Explain your answer.
   Grade: /5

A SSH and File Transfer Workflow

This appendix explains how to connect to the Raspberry Pi from a Windows PC, transfer the Module 3 controller files, prepare the Python environment, and download the result CSV files after each run.

Throughout this appendix, `team-x` and `10.107.151.137` are used as examples only. Replace them with your own Raspberry Pi username and IP address.

Step 1: Connect the Windows PC and Raspberry Pi to the Same Network

Connect your Windows PC and Raspberry Pi to the same hotspot as used in Section 4. SSH and file transfer will only work if both devices are on the same network.

Step 2: Find the Raspberry Pi IP Address

On the Raspberry Pi, open a terminal and run:

```
hostname -I
```

The first IP address shown is usually the Raspberry Pi address on the current network. Write it down, because it will be used for SSH and `scp` commands.

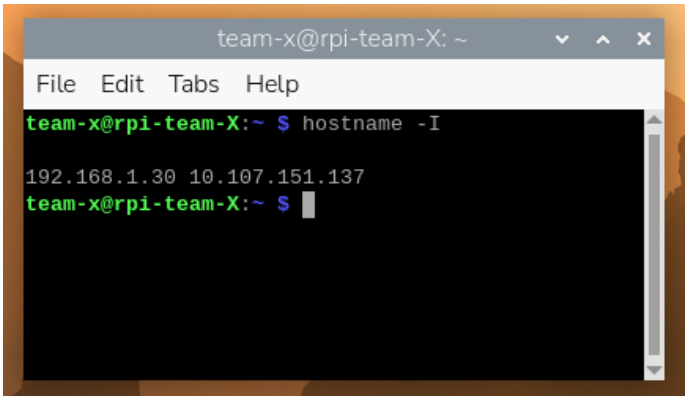


Figure 31: Retrieving the Raspberry Pi IP address using `hostname -I`.

Step 3: Test the Connection from Windows

On the Windows PC, open **Command Prompt** or **PowerShell** and run:

```
ping 10.107.151.137
```

Replace `10.107.151.137` with your Raspberry Pi IP address. A successful reply confirms that the Windows PC can reach the Raspberry Pi.

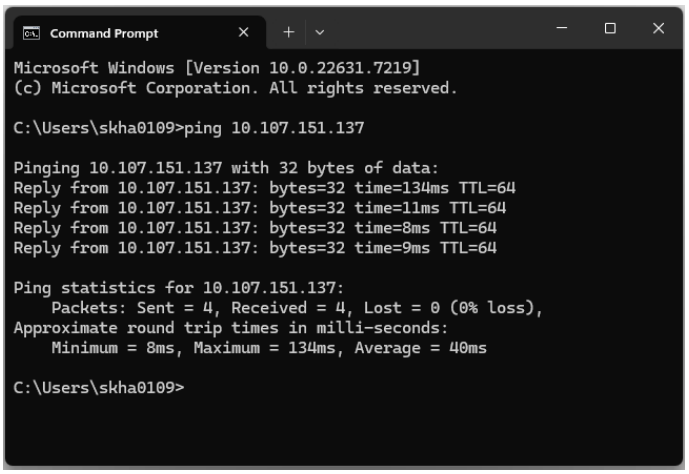


Figure 32: Successful ping to the Raspberry Pi from Windows.

Step 4: Enable SSH on the Raspberry Pi

On the Raspberry Pi, enable and start the SSH service:

```
sudo systemctl enable ssh
sudo systemctl start ssh
```

This allows the Windows PC to open a terminal session on the Raspberry Pi.

Step 5: Connect to the Raspberry Pi from Windows

From Windows Command Prompt or PowerShell, run:

```
ssh team-x@10.107.151.137
```

Replace `team-x` with your team username and replace `10.107.151.137` with your Raspberry Pi IP address. Enter the Raspberry Pi password when prompted.

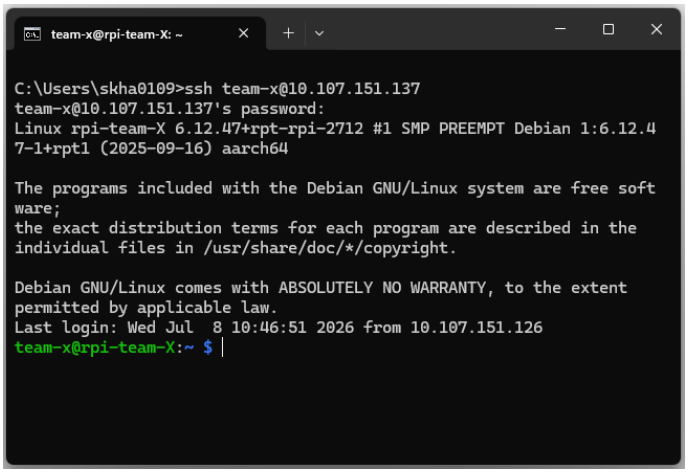


Figure 33: Successful SSH connection to the Raspberry Pi from Windows.

Step 6: Create the Module 3 Working Directory

After logging in to the Raspberry Pi through SSH, create a working directory for Module 3:

```
mkdir -p ~/toy
```

The command creates a folder named `toy` in the Raspberry Pi home directory. If the folder already exists, the command will not produce an error.

Step 7: Transfer the Files and Dataset to the Raspberry Pi

On the Windows PC, open PowerShell in the folder that contains the Module 3 files. Then copy the QP script, MPC script, and dataset to the Raspberry Pi:

```
scp toy_qp_modbus_node646_students.py toy_mpc_modbus_node646_students.py
    toy_example_N646_students.csv actual_N646_students.csv team-x@10.107.151.137:/home/
    team-x/toy/
```

Replace `team-x` and `10.107.151.137` with your own team username and Raspberry Pi IP address.

Step 8: Prepare the Python Environment on the Raspberry Pi

In the SSH session, move into the Module 3 folder, create a Python virtual environment, activate it, and install the required packages:

```
cd ~/toy
python -m venv .venv
source .venv/bin/activate
pip install pymodbus cvxpy pandas numpy
```

When the virtual environment is active, the terminal prompt should show `(.venv)`. You are now ready to run the QP and MPC controllers.

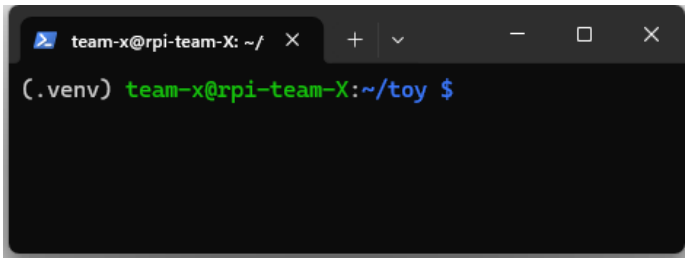


Figure 34: Activated Python virtual environment on the Raspberry Pi.

Step 9: Download Result CSV Files from Raspberry Pi to Windows

After completing the QP or MPC run, download the result CSV files from the Raspberry Pi to the Windows PC. In a Windows PowerShell window, open the folder where you want to save the results and run:

```
scp team-x@10.107.151.137:/home/team-x/toy/*.csv .
```

The final dot `.` means "save the files in the current Windows folder" and the command downloads the generated controller CSV files.

B Measuring Feeder Power at Node 632 using a Power Meter

This appendix describes how to add a **Power Meter** and **Probe** inside the Node 632 subsystem of the MIEEE-13NF model to measure the feeder active power. Node 632 sits at the head of the feeder, so the average active power measured here corresponds to the total active power delivered to the feeder. The resulting signal, `Node 632.Probe1`, is displayed in HIL SCADA using a **Trace Graph** widget and produces the feeder active-power plots.

Objective

Add a Power Meter to the Node 632 schematic, wire its three phase voltages (V_a, V_b, V_c) and three phase currents (I_a, I_b, I_c), and log the average active power (P_{avg}) through a Probe so that the feeder active power can be viewed in HIL SCADA.

Step 1: Open the Node 632 Subsystem

1. Single-click **Node 632** to select it.
2. With Node 632 selected, press `Ctrl` + `Enter` to enter (open) the Node 632 subsystem schematic, as shown in Figure 35.

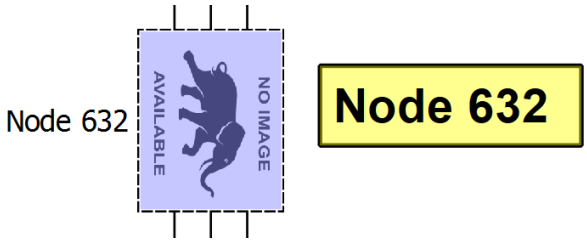
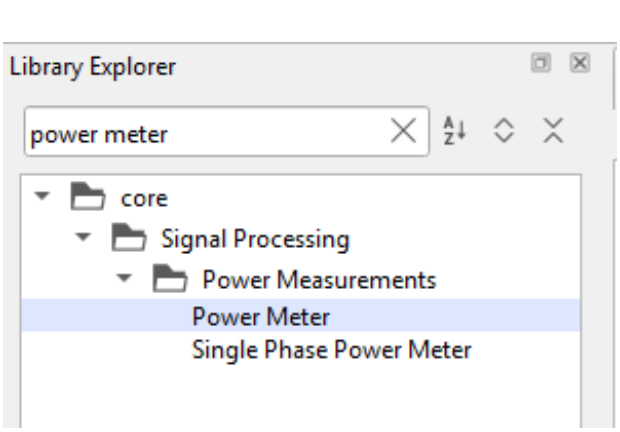


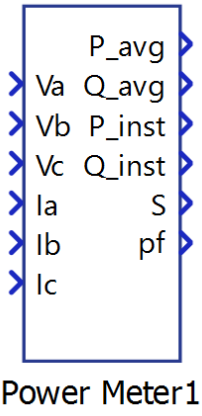
Figure 35: Node 632 selected before pressing `Ctrl` + `Enter` to open the subsystem.

Step 2: Add the Power Meter

1. In the **Library Explorer**, search for `power meter` and navigate to `core` → `Signal Processing` → `Power Measurements` → `Power Meter`, as shown in Figure 36a.
2. Drag the **Power Meter** onto the Node 632 schematic, as shown in Figure 36b.



(a) Locating the Power Meter in the Library Explorer.



(b) Power Meter dragged into the schematic.

Figure 36: Adding the Power Meter block: (a) Library Explorer and (b) placement in the Node 632 schematic.

Step 3: Connect the Phase Currents

1. Connect `I1` to `Ia`, as shown in Figure 37. **Note:** the Power Meter is rotated in this figure.
2. Similarly, connect `I2` to `Ib` and `I3` to `Ic`.

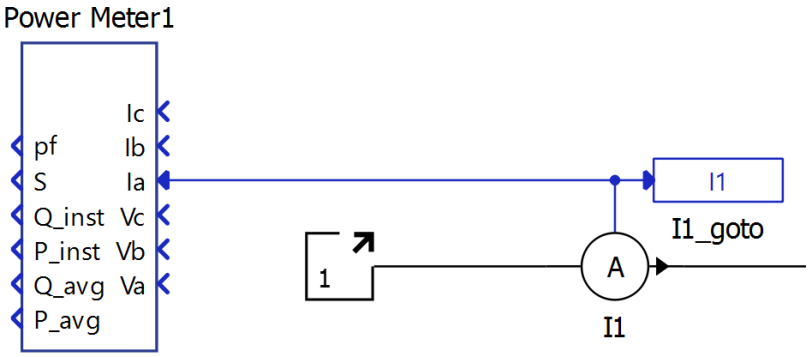


Figure 37: Connecting current `I1` to the Power Meter input `Ia`. The Power Meter is rotated in this figure.

Step 4: Connect the Phase Voltages

1. Connect `V1` to `Va`, `V2` to `Vb`, and `V3` to `Vc`, as shown in Figure 38.

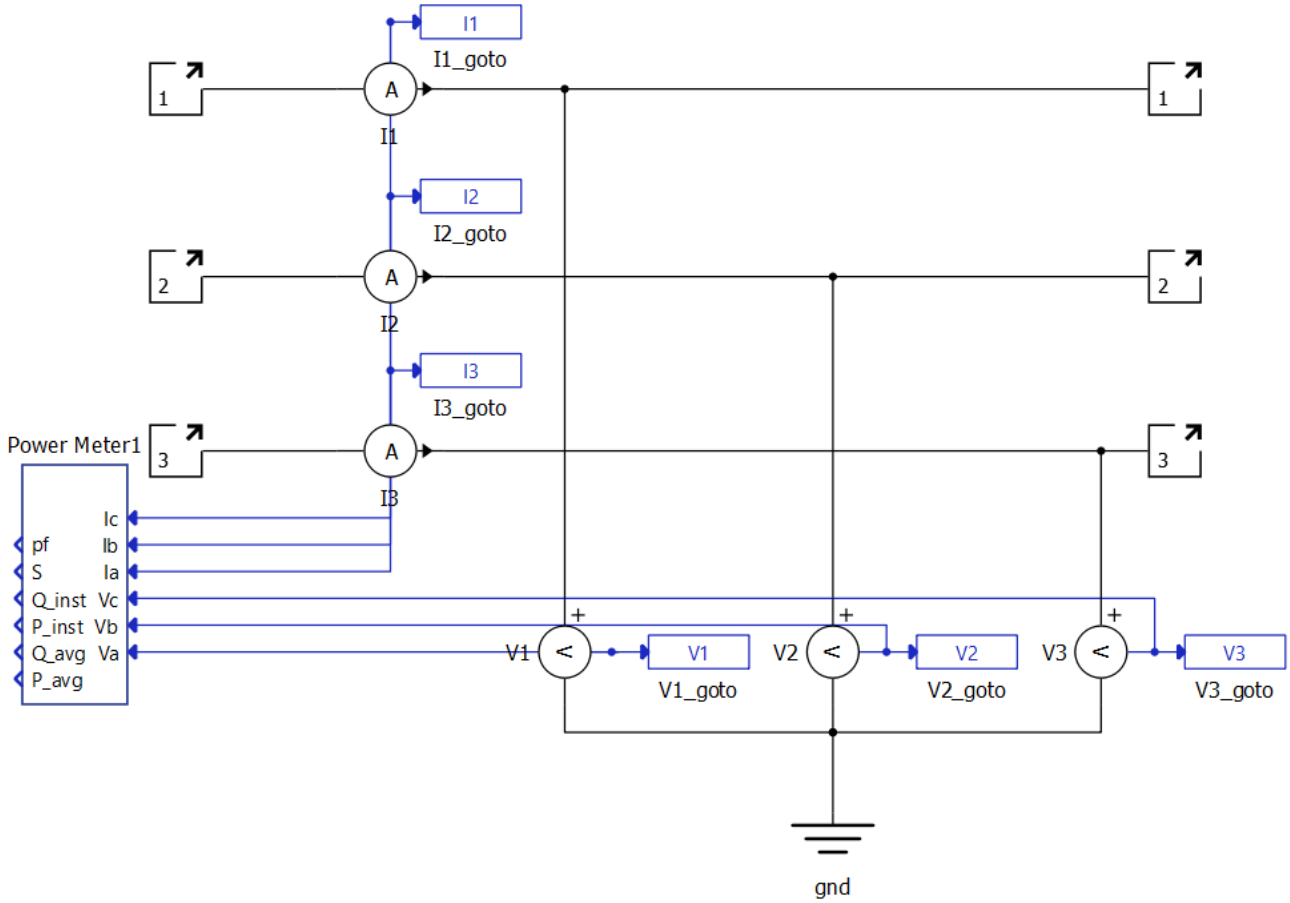
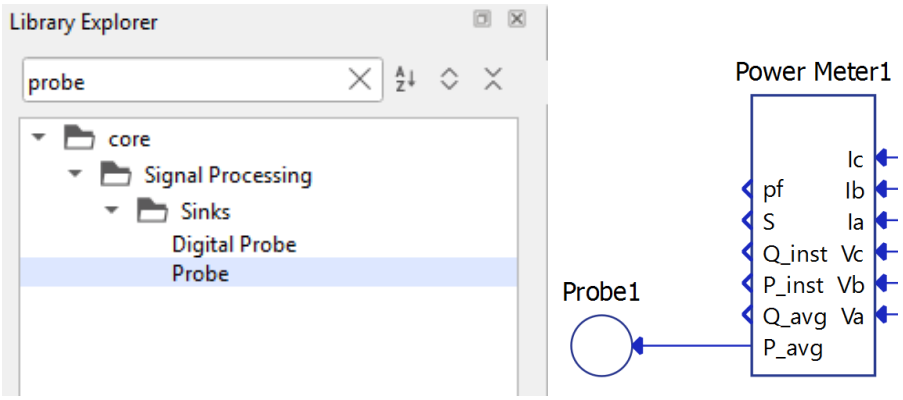


Figure 38: Power Meter with all three phase currents (I_a, I_b, I_c) and all three phase voltages (V_a, V_b, V_c) connected.

Step 5: Add and Connect the Probe

1. In the **Library Explorer**, search for `probe` and navigate to `core` → `Signal Processing` → `Sinks` → `Probe` as shown in Figure 39a. Drag a **Probe** onto the schematic.
2. Connect the Probe to the `P_avg` output of the Power Meter, as shown in Figure 39b.



(a) Locating the Probe in the Library Explorer.

(b) Probe connected to `P_avg`.

Figure 39: Adding the Probe and connecting it to the average active-power output of the Power Meter.

Step 6: Configure the Probe

1. Double-click **Probe1** to open its properties.
2. Check **Override signal name**.
3. Set **Signal name** to `P` and **Signal type** to `active power`.
4. Leave **Signal streaming** unchecked and **Execution rate** as `inherit`, as shown in Figure 40.

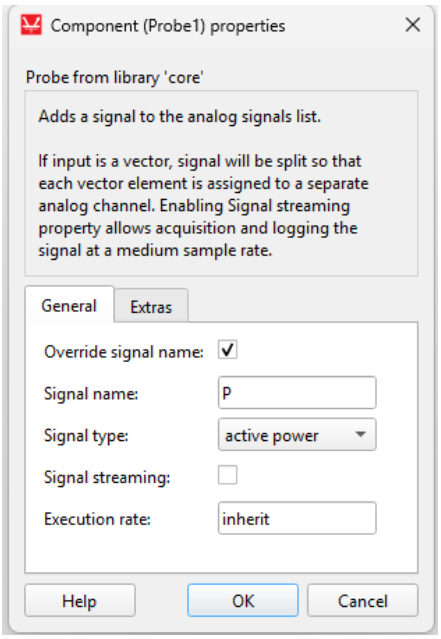


Figure 40: Probe1 properties: overriding the signal name to `P`, signal type `active power`, signal streaming unchecked, and execution rate `inherit`.

Step 7: Compile and (Re)load the Model

1. Click **Compile and (re)load model in HIL SCADA** in HIL SCADA.

Expected Validation Error: Execution Rate Mismatch

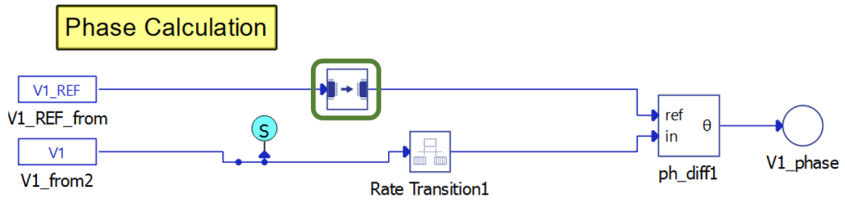
After compiling, you will be prompted with:

> *Validation was not completed successfully! Component Node 632.Power Meter1 can't inherit execution rate since its input components have different execution rates.*

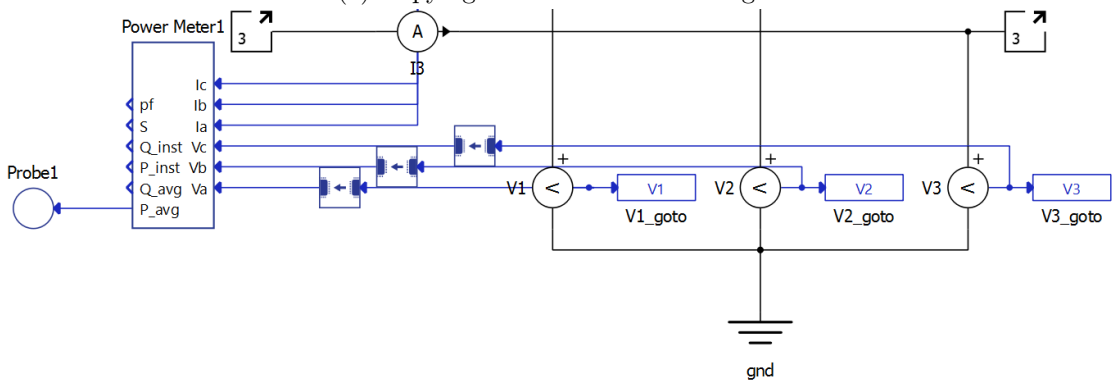
This occurs because the voltage inputs (V_a, V_b, V_c) and current inputs (I_a, I_b, I_c) feeding the Power Meter run at different execution rates, so the Power Meter cannot resolve a single inherited rate. Step 8 resolves this by inserting a **CPU Transition** widget on the voltage inputs.

Step 8: Resolve the Error with CPU Transition Widgets

1. Copy a **CPU Transition** widget, as shown in Figure 41a.
2. Paste and connect a CPU Transition between `V1` and `Va`, between `V2` and `Vb`, and between `V3` and `Vc`, as shown in Figure 41b. **Note:** the Power Meter and CPU Transition widgets are rotated in this figure.
3. Recompile and (re)load the model. Validation should now complete successfully.



(a) Copying a CPU Transition widget.



(b) CPU Transition widgets connected on the voltage inputs. The Power Meter and CPU Transition widgets are rotated in this figure.

Figure 41: Inserting CPU Transition widgets between the voltage sources and the Power Meter voltage inputs to align execution rates. The Power Meter and CPU Transition widgets are rotated in (b).

Step 9: View the Feeder Power in HIL SCADA

1. In the **Trace Graph** widget, add the `Node 632.Probe1` signal, as shown in Figure 42, to view the average active power of Node 632.

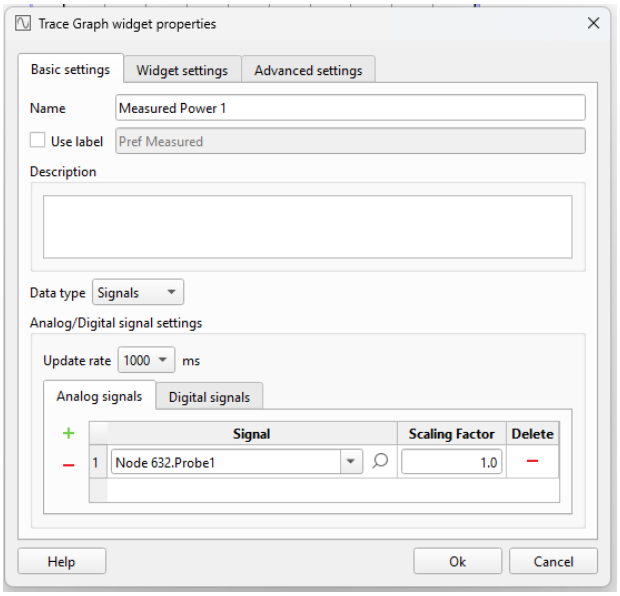


Figure 42: Adding the `Node 632.Probe1` signal to the Trace Graph widget in HIL SCADA to display the feeder average active power.

C Troubleshooting: Raspberry Pi Not Appearing in Raspberry Pi Connect

This appendix provides troubleshooting steps for cases where the Raspberry Pi does not appear in the Raspberry Pi Connect device list after the Pi has been flashed and Raspberry Pi Connect has been configured.

Symptoms

- The Raspberry Pi is connected to the phone hotspot or local network.
- The laptop is connected to the same network.
- The laptop cannot ping the Raspberry Pi using its expected hostname.
- The Raspberry Pi does not appear in the Raspberry Pi Connect device list.

Resolution

1. Connect the Raspberry Pi to a monitor, keyboard, and mouse so that it can be accessed locally.
2. Open a terminal on the Raspberry Pi and determine its current IP address:

```
hostname -I
```

   For example, the command may return:

```
172.20.10.5
```

3. On the laptop, open a terminal or PowerShell window and connect to the Raspberry Pi using SSH and its IP address instead of its hostname:

```
ssh team-01@172.20.10.5
```

   Replace `team-01` with your Raspberry Pi username and `172.20.10.5` with the IP address obtained in the previous step.

4. Once connected, verify the Raspberry Pi hostname, username, and IP address:

```
hostname
whoami
hostname -I
```

   For example, these commands may confirm:

   - Hostname: `rpi-team-01`
   - Username: `team-01`
   - IP address: `172.20.10.5`

5. Check the status of Raspberry Pi Connect:

```
rpi-connect status
```

6. If Raspberry Pi Connect is not enabled or the Raspberry Pi has not been linked to an account, enable the service and begin the sign-in process:

```
rpi-connect on
rpi-connect signin
```

   Follow the displayed sign-in instructions to link the Raspberry Pi to the required Raspberry Pi Connect account.

7. After the linking process is complete, refresh Raspberry Pi Connect on the laptop. The Raspberry Pi should now appear in the device list.

Note: If the Raspberry Pi can be accessed successfully using its IP address but not using the `.local` hostname, the issue is likely related to hostname discovery on the local network rather than the Raspberry Pi's network connection itself.